Voice Life Manager App Blueprint

Complete implementation design: frontend · backend · through launch (v1.1)

Date: 2026-10-02 (KST) · v1.1: Basic plan price KRW 2,900, expanded friend display-name feature, added rationale for AI model choice

Intended readers: frontend owner (app owner), backend owner (Supabase), launch owner

Scope: all features, data model, API, screens, rules, exceptions, and schedule for the first release (App Store · Play Store)

Basis: requirements-finalization conversations with the app owner (2026-10-01 to 10-02) and review of the current repository code

Table of Contents

At a Glance 3

0. How to Read This Document · Glossary 5

1. Product Overview · User Tiers · Feature Matrix 7

2. Overall Structure (Architecture) 9

3. Accounts · Login · Onboarding · Backup · Account Deletion 11

4. Input: Voice · Text · AI Interpretation Rules 15

5. Friends: Adding · Managing · Profile Photos 19

6. Sharing: Sending and Receiving Events · Expenses 22

7. Visibility · Viewing Friends' Schedules 26

8. Notifications 27

9. Subscriptions · Trials · Limits · Payments · Costs 29

10. Settings · Font Size · Excel Export 31

11. Screen Design (Frontend Screen List and Flows) 33

12. Frontend Implementation Design 36

13. Backend Implementation Design (Supabase) 39

14. AI Gateway and Interpretation Contract 47

15. Exception · Error Handling Table 51

16. Security · Privacy 53

17. Store Release Requirements 54

18. Development Order · Roles · Definition of Done 55

19. Excluded from v1 · Candidates for v2 56

20. Launch Prerequisites · Items to Verify 57

Appendix A. Full Decision Log 58

At a Glance

This page collects only the essentials so that reading it alone gives you the full picture of the app. See each chapter for detailed rules.

What the app does

- Automatically records expenses and events from a single spoken sentence
- Lets you add events and expenses on behalf of friends and family
- Parents who set "항상 넣기" ("Always add") get hospital appointments sent by their children added automatically, complete with reminders
- Korean only; simultaneous Android and iOS release

Pricing

- Guest: records stay on the phone only (no backup, friends, or AI)
- Free member: backup, friends, receiving, 3 AI trial uses, 1 sharing trial
- **Basic KRW 2,900**: 150 AI uses/month · 20/day, sending
- **Plus KRW 7,900**: 500 AI uses/month · 40/day, sending

Core sharing rules

- Sending requires a subscription; anyone (logged in) can receive
- The recipient chooses per friend: Ask every time / Always add / Always review only / Don't receive
- The sender only sees "Pending delivery · Delivered" and never learns whether it was accepted
- Edits and cancellations apply immediately; items the recipient deleted are never revived
- Friends' names can be renamed freely, shown only on your own screen

Accounts · Data

- Apple · Google · Kakao login; can start as a guest
- The phone is the source of truth; logging in enables server backup → restore even after uninstalling
- One device per account; account deletion is immediate
- Speech is recognized on-device; recordings are not stored

Tech stack

- App: Expo SDK 57 · React Native · TypeScript · SQLite
- Server: Supabase (Postgres + RLS, Auth, Storage, Edge Functions)
- AI: OpenAI gpt-4.1-mini, about KRW 1.3 per call
- Push: Expo Push; Payments: store in-app purchases + server verification

Development order

- M0 Setup → M1 Local complete → M2 Accounts · Backup
- M3 AI expansion → M4 Friends · Sharing → M5 Subscriptions
- M6 Store launch preparation
- Recheck "Verify" items before launch (Chapter 20)

0. How to Read This Document · Glossary

0.1 Labels

Label	Meaning
Confirmed	Decided directly by the app owner, or approved by the owner as "go with the recommendation." Changing it requires the app owner's approval.
Design	A technical detail decided by this document to implement the confirmed items. The owner of the area may change it if there is a better way to achieve the same result, but the change must be recorded in this document.
Verify	A fact that could not be verified when this document was written, such as external policies, prices, or laws. It must be checked against the latest official sources before implementation or launch.

0.2 Glossary

Term	Description
Guest	A user who has not logged in. Records stay on the phone only; no backup, friends, or AI.
Free member	A user who has logged in but has not subscribed. Has backup, friends, receiving, viewing friends' schedules, and trials (3 AI uses · 1 share).
Subscriber	A user on the Basic (KRW 2,900) or Plus (KRW 7,900) plan.
Local DB	The SQLite database on the phone. It is the **source of truth** for personal records.
Backup	A copy of the personal records in the local DB kept on the server (Supabase). Used to restore after the app is deleted and reinstalled.
Send / shared item	An event or expense a subscriber sends to a friend. Stored on the server as `shared_items`.
Delivery	The unit of one shared item going to one recipient. States: Pending delivery → Delivered / Expired / Cancelled.
Delivered	The state in which the recipient's app has fetched the item from the server, saved it on the phone, and sent an acknowledgment (ack) to the server. Whether the recipient "added it" or "only reviewed it" is not visible to the sender.
Receive mode	How the recipient handles incoming items, set per friend and per type (event/expense): Ask every time, Always add, Always review only, Don't receive.
Inbox	The list where received items handled as "review only" stay for 30 days. They can be added to the calendar or expenses later.
Deletion record (tombstone)	A "deleted" marker kept so that an item the recipient deleted is not revived by the sender's later edits.

ActionProposal	A "here is what I will save" proposal produced from input (by the local parser or AI). It is actually saved only through the sequence Validator → Executor → Repository → local DB.
RLS	Row Level Security. Rules by which Postgres enforces, row by row, "who can read and write this row." Permission checks are done by the DB, not the app.
Edge Function	A server function running on Supabase (Deno/TypeScript). Handles work that needs secret keys, such as payment verification, link handling, and AI calls.
outbox (send queue)	A local table that queues, in order, changes to be sent from the phone to the server. Changes accumulate while offline and are sent once a connection is available.
KST	Korea Standard Time (UTC+9). The reference for resetting daily limits.

0.3 Common Rules for Time · Money Confirmed

- All timestamps are stored in the DB and on the server as **UTC ISO 8601** and displayed in the device time zone. Events also store their time zone (IANA, e.g. `Asia/Seoul`).
- Amounts are stored as **positive integers (won)**, and expense / income are distinguished by `type`. Negative amounts are not used. A currency code (`KRW`) field is included.
- All IDs are UUIDs. Core entities have `created_at` and `updated_at`, and soft deletion (`deleted_at`) is used where recovery is needed.
- A failed save, notification scheduling, or send is never shown as a success. Partial success is shown as partial success.
- Korean is the only supported language.

1. Product Overview · User Tiers · Feature Matrix

1.1 One-line Definition

A Korean-language life management app that **automatically records expenses and events** from a single spoken sentence (or a short text) and lets you **add events and expenses on behalf of** friends and family. Example: when a child sends "내일 오후 3시 아버지 병원 예약" ("Dad's hospital appointment tomorrow at 3 PM"), it goes into the father's phone calendar and the reminder goes off too.

1.2 Key Usage Scenarios

1. "밥 7000원" ("meal 7,000 won") → an expense of KRW 7,000 (Food) is saved immediately (available to free users too).
2. "밥 7000원짜리 먹었고 오늘 친구랑 오후7시 약속있다" ("Had a 7,000-won meal, and I have plans with a friend today at 7 PM") → 1 expense + 1 event are saved in one go (subscription · AI).
3. "내일 오후 7시에 강남역에서 만나기 이걸 민수에게 줘" ("Meet at Gangnam Station tomorrow at 7 PM — give this to Minsu") → after a confirmation screen, the event is sent to Minsu (subscription).
4. If parents have set "항상 넣기" ("Always add"), hospital appointments sent by their children are automatically added to the parents' calendar and a reminder goes off at the set time.
5. Friends can see each other's public events (available to free members too).

1.3 Feature Matrix by User Tier Confirmed

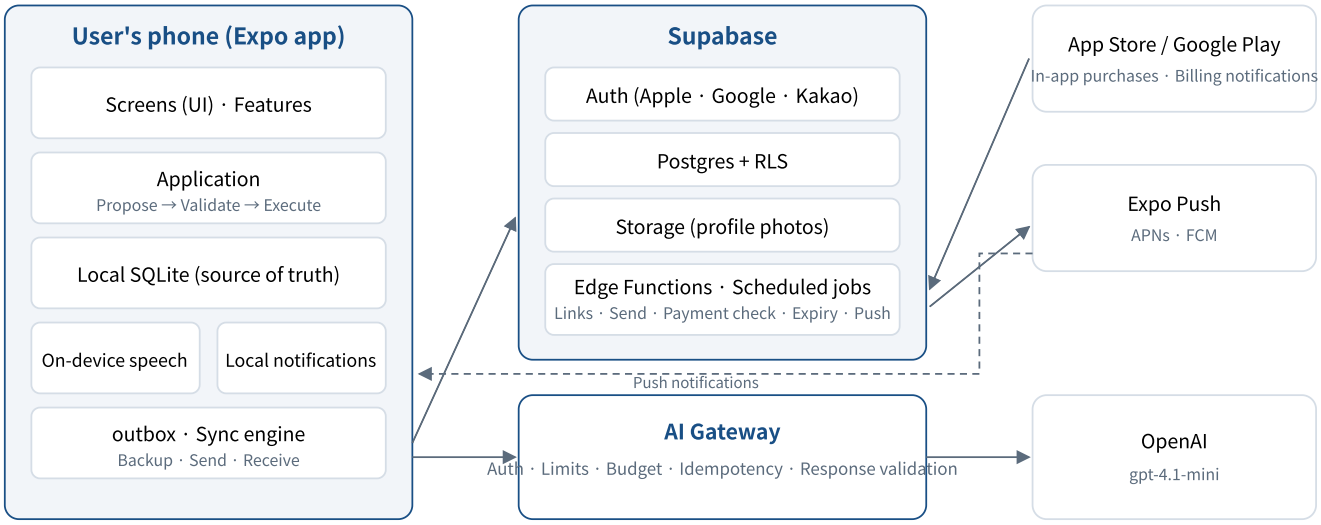
Feature	Guest	Free member	Basic subscription	Plus subscription
Manual entry of expenses · events (forms)	O	O	O	O
Local processing of simple expense sentences ("밥 3000원" ("meal 3,000 won"))	O	O	O	O
Voice input (on-device recognition)	O (only sentences that can be processed locally)	O (local + trial)	O	O
AI interpretation (compound sentences, event sentences)	X (prompt to log in)	3 trial uses (lifetime)	150/month · 20/day	500/month · 40/day
Server backup · restore on reinstall	X	O	O	O
Adding friends · viewing friends' public events	X	O	O	O
Receiving events · expenses	X	O	O	O

Sending events	X	1 trial (1 recipient, events only)	O (50 per day)	O (50 per day)
Sending expenses (expense requests)	X	X	O	O
Sending by voice	X	Trial (uses 1 AI + 1 share)	O	O
Excel export	O	O	O	O
Profile photo	X	O	O	O
Font size · notification settings	O	O	O	O

> The plan name "Unlimited" is not used **Confirmed** . Without a cost cap there is a risk of losses, and if the store description differs from the actual limits there is a risk of review rejections and complaints. Limits are stated as numbers.

2. Overall Structure (Architecture)

2.1 Components



2.2 Data Source-of-Truth Rules Confirmed Design

Data	Source of truth	What lives on the server
My events · expenses · to-dos · reminder settings	Phone (local DB)	Backup copy (logged-in users only). Events visible to friends also have a separate "public events" copy.
Raw input text (`rawInput`), action log (ActionLog), temporary drafts	Phone	None. Also excluded from backup.
Voice (audio)	Not stored	None. Deleted immediately once recognition finishes.
Account · nickname · profile photo · friend relationships · receive settings	Server	Source of truth
Sent items · delivery status · deletion records	Server	Source of truth. The recipient's phone stores the received copy.
Subscription status · usage · trial counts	Server	Source of truth (the app only keeps a display cache)

2.3 Tech Stack

Area	Choice

App	Expo SDK 57, React Native 0.86, TypeScript, Expo Router, expo-sqlite, expo-speech-recognition (on-device recognition, `ko-KR`). Supports Android and iOS simultaneously.
Backend	Supabase: Postgres, Auth, Storage, Edge Functions, scheduled jobs (pg_cron, etc.) **Confirmed**
AI	OpenAI `gpt-4.1-mini-2025-04-14` (current gateway setting). Responses API + strict JSON schema.
Push	Expo Push service (via APNs · FCM) **Design**
Payments	App Store in-app purchases · Google Play Billing. Receipts verified on the server. Whether to use a service such as RevenueCat is the backend owner's decision (see 13.9).
Build · Submission	EAS Build / EAS Submit **Design**

> The existing repository's `AGENTS.md` contains scope rules from an earlier phase, such as "core data is local-only, no cloud sync, no recurring events." The app owner has decided that the design in this document takes precedence **Confirmed** . Before starting implementation, revise **only the product scope rules** in `AGENTS.md` to match this design, and keep the technical rules (UTC storage, positive-integer amounts, undo, idempotency, never reporting failure as success, layer dependency direction). If it is not revised, AI coding tools used in the repository may treat server-storage work as "forbidden."

3. Accounts · Login · Onboarding · Backup · Account Deletion

3.1 Login Methods and Timing Confirmed

- Login methods: **Apple, Google, Kakao** (all included in the first release). There is no email or phone-number login.
- Users can **start right away as a guest**. Login is required when the user tries to use backup, friends, AI, or subscriptions.
- The login session is persisted (auto-login).
- An Apple account and a Google account are separate accounts. Account linking (binding two methods to one account) is not supported in v1. The login screen help explains that switching platforms does not carry over backups and subscriptions.
- Verify The scope of Supabase Auth's Kakao provider support, and the Kakao Developers app registration procedure (request only nickname and unique ID so that email collection is not required).
- Verify App Store review guidelines: offering third-party social login requires offering Sign in with Apple (or an equivalent privacy-preserving login). Since Apple login is included, this is expected to be satisfied.

3.2 First-Launch Flow Confirmed

1. Welcome screen (one-page app introduction).
2. **Notification permission explanation screen**: "일정 시간과 친구가 보낸 일정을 알려 드리려면 알림이 필요해요." ("We need notifications to remind you of event times and events your friends send.") [계속] ("Continue") → system notification permission request. The app remains usable even if denied.
3. Login choice screen: [애플로 계속] ("Continue with Apple") [구글로 계속] ("Continue with Google") [카카오로 계속] ("Continue with Kakao") [로그인 없이 시작] ("Start without logging in").
4. If logged in (first sign-up): set nickname → choose default visibility for new events (public / private) → Home. For accounts that already have a profile (reinstall · device change), nickname and visibility setup are skipped, and if a server backup exists it is restored (if guest records also exist, the choice screen from 3.3 appears) → Home.
5. Guest: straight to Home. A notice "백업 꺼짐 · 로그인하면 앱을 지워도 기록이 안전해요" ("Backup off · Log in to keep your records safe even if you delete the app") is shown persistently at the top of Home.
6. Microphone permission is requested the first time the voice button is pressed.

> On iOS, once notification permission is denied, the app cannot show the system prompt again. So an explanation screen is shown before the system prompt, and when permission is denied the settings screen shows a "알림 꺼짐 · 설정 열기" ("Notifications off · Open Settings") notice. The existing `PERMISSION_BLOCKED` is used for the local notification state.

3.3 Guest → Login Transition Design

- When a guest has saved 3 or more records, a one-time "로그인하면 백업돼요" ("Log in to back up your records") prompt is shown (once dismissed it is not shown again; the Home banner remains).
- When the user speaks a sentence that requires AI, a "로그인하면 AI 체험 3회를 쓸 수 있어요" ("Log in to get 3 free AI trial uses") screen is shown. The same login prompt is shown when the user taps friend features.
- If the server has **no** backup for that account at login: the guest records on the phone become that account's data as-is, and a full backup is uploaded.
- All events and to-dos created as a guest are stored as **private**; even if the user picks "public" as the default visibility after logging in, **past items are not made public in bulk**. The default applies only to items newly created after login.
- If the server **has** a backup at login and the phone also has guest records, a choice screen is shown:
  - [서버 백업 불러오기] ("Load server backup") — deletes the guest records on the phone and restores from the server backup.
  - [이 폰 기록으로 덮어쓰기] ("Overwrite with this phone's records") — deletes the server backup and uploads the phone's records.
  - Shows text stating that neither choice can be undone, along with the record count on each side. Merging is not offered in v1.

3.4 Nickname Confirmed

Item	Rule
Length · characters	2–12 characters; Korean, English letters, and digits only. No spaces or special characters.
Uniqueness	Must be unique across all users. Uppercase and lowercase English letters are treated as the same.
Banned words	The server checks against a banned list of profanity and operator impersonation (e.g. "관리자" ("administrator"), "운영자" ("operator"), the app name). The list is maintained by the operators.
Changes	Once every 30 days. The next date a change is allowed is shown on screen.
Released nicknames	A nickname released through a change or account deletion cannot be used by anyone else for 30 days (to prevent impersonation).
Searchability	A "닉네임으로 나를 찾을 수 있게 하기" ("Let others find me by nickname") toggle in settings (on by default).

3.5 One Account, One Device Confirmed

- An account can be logged in on only one device at a time. Logging in on a new device logs out the previous device.
- Right after logging in on a new device: show the notice "이전 기기의 마지막 백업: 10월 2일 15:30. 그 이후 이전 기기에서 바꾼 내용은 옮겨지지 않을 수 있어요." ("Last backup from your previous device: Oct 2, 15:30. Changes made on the previous device after that may not be transferred."), then restore.

- On its next server communication, the previous device receives `DEVICE_REPLACED`, shows "다른 기기에서 로그인했어요" ("You logged in on another device"), deletes local data, and returns to the guest first screen. Design

3.6 Server Backup · Restore Confirmed

- Applies to: all logged-in users (including free).
- Approach: **the phone is the source of truth; the server holds a backup copy**. Changes are queued in the outbox and uploaded immediately when online (batched over at most a few seconds), or after reconnecting when offline.
- Backed up: expenses · income, events, to-dos, reminder settings, app settings (font size, notification settings, sending defaults, visibility defaults), copies of received items and the inbox, deletion records.
- Excluded from backup: raw input text (`rawInput` is `null` in the backup), ActionLog, conversation context, temporary drafts, audio. As a result, there is no "undo" history after a restore.
- Conflict rule: since there is one device per account, for each entity the one with the more recent `updated_at` wins. Design
- Restore flow: login → check whether a backup exists → "Restoring (progress)" screen → write to the local DB (via the repository layer) → **reschedule all upcoming reminders** → done. On failure, show a "Try again" button and the number of failures.
- After a restore, notification and microphone permissions must be granted again for each installation, so the permission flow in 3.2 is repeated.
- Settings > My Data shows "Last backup time" and "N items waiting to back up," and shows failures as failures.

3.7 Comparison: Unfriend · Log Out · Delete App · Delete Account · Reset Confirmed

Action	Server	My phone	Other person's phone
Disconnect friend	Only the connection is removed. Exchanged items and deletion records are kept, so reconnecting picks up where things left off.	Received items kept	Received items kept
Log out	No change (backup kept)	Local data **deleted**. If there are changes not yet backed up, warn before logging out and attempt the upload first.	No change
Delete app (logged-in user)	No change	Deleted by the OS. Restored by logging in after reinstalling.	No change
Delete app (guest)	N/A	Records are lost and cannot be recovered.	N/A

Delete account	**Immediately** deletes account · friend relationships · originals of sent items · backup · profile photo · nickname (only the 30-day nickname reservation remains)	App data deleted, back to the first screen	Items I sent remain on the other person's phone, with the sender changed to "탈퇴한 사용자" ("Deleted user"). Items not yet delivered are cancelled.
Reset (a) one person	Disconnects from that friend, deletes delivery records exchanged with that friend (pending deliveries are cancelled first), deletes copies received from that friend in the backup. Reconnecting starts fresh from scratch	Items received from that friend deleted	Items on the other person's phone belong to them and are kept
Reset (b) all shared data	Deletes all friend relationships, sent-item records, received delivery records, invite links, and public event copies. Also deletes received-item copies, the inbox, and deletion records in the backup (so they are not revived on restore). Account · personal record backup · subscription are kept.	All received items deleted, friend list emptied	Kept
Reset (c) all app data	Deletes my entire backup and public event copies (account and friend connections are kept)	All personal records deleted	Kept

- Account deletion and resets all require **two-step confirmation**: step 1 shows a list of "what will be deleted"; step 2 requires typing a word such as "탈퇴" ("delete account") or a long press. Design
- Guests only have reset (c), which deletes only phone data.
- The account deletion screen shows the notice "구독은 앱에서 해지되지 않아요. 앱스토어 / 플레이스토어 구독 관리에서 해지하세요." ("Your subscription is not cancelled in the app. Cancel it in App Store / Play Store subscription management.") with a button to go there.
- Minimum age: 14 or older. On the login screen, when agreeing to the Terms and Privacy Policy, the user also confirms "만 14세 이상입니다" ("I am 14 years of age or older"). Verify Legal review of whether this approach satisfies the legal verification obligation.
- Re-registering is treated as a new account and grants the 3 AI trial uses and 1 sharing trial again.

4. Input: Voice · Text · AI Interpretation Rules

4.1 Input Paths

1. **Voice**: the sentence is obtained via on-device speech recognition (`requiresOnDeviceRecognition: true`, `ko-KR`, no recording files saved). If the device does not support on-device recognition, the voice button is disabled and the notice "이 기기에서는 글자로 입력해 주세요" ("Please type your input on this device") is shown. Server-side speech recognition is not used as a fallback. Confirmed
2. **Text**: the same sentence input field.
3. **Manual entry forms**: expense form (amount, category, memo, date · time), event form (title, date, time or approximate time, location, reminder, visibility).

4.2 Processing Order (Router)

```
Sentence input
 ├─ Can the local parser handle it? (single expense, exactly one "원" amount, no date or companion words)
 │    └─ Yes → create expense proposal → validate → auto-save → result card + [Undo]
 ├─ No → AI required
 │    ├─ Guest → "Log in for 3 AI trial uses" prompt → [Log in] / [Enter manually]
 │    ├─ Free member, trials remaining → "N AI trial uses left. Use one?" → AI call
 │    ├─ Free member, no trials left → "AI is required" notice → [View subscriptions] / [Enter manually (sentence pre-filled into the title · memo field)]
 │    └─ Subscriber, limit remaining → AI call (when over the limit: "You've used up this month's / today's limit" + manual entry)
 └─ If offline, do not call AI; keep the sentence as a "pending draft" → after reconnecting, the user taps [Process again] manually
```

- Current scope of the local parser (verified in the repository code): exactly one "원" amount; rejects income, negation, planning, and question expressions; hands off to AI if there are date/time words ("오늘" ("today"), "7시" ("7 o'clock"), etc.) or companion words ("친구" ("friend"), "랑" ("with"), "같이" ("together"), etc.). It does not process sentences with two or more amounts. Therefore, free users must say expenses one at a time.
- Events from free users are taken via the **manual entry form**. A local rule-based parser for events will not be built in v1 Confirmed .
- Saves resulting from AI calls also use auto-save + [Undo]. However, **any action that sends something to another person always goes through a confirmation screen** Confirmed .
- Actions that need AI while offline are never executed automatically later without the user's knowledge. Destructive actions that affect other people (cancelling a sent item, disconnecting · blocking a friend, reset, account deletion) are disabled with a notice while offline. Deletions on my own phone (including deleting received items) happen immediately, and only the deletion record that must be reported to the server is put in the outbox.

4.3 Compound Sentence Handling Confirmed

When a single sentence contains multiple actions, the AI returns **multiple actions within one proposal**. Only one AI call is deducted.

- Each action is validated and saved separately. Independent actions are saved first even if another action is waiting on a question.
- An action that depends on another action (e.g. an event's "1 hour before reminder") waits until that event is finalized.
- The result screen shows a card per action and distinguishes the status: Saved / Awaiting answer / Failed.

Example sentence	Result
밥 7000원짜리 먹었고 오늘 친구랑 오후7시 약속있다 ("Had a 7,000-won meal, and I have plans with a friend today at 7 PM")	Card 1: expense KRW 7,000 · Food · memo "밥" ("meal") · current time (Saved). Card 2: event "친구랑 약속" ("Plans with a friend") · today 19:00 (Saved). The default reminder settings apply. Both cards have [Undo].
7시에 밥 7000원 먹었어 ("Had a 7,000-won meal at 7") (20:00)	Expense today 19:00
친구랑 19시 약속 ("Plans with a friend at 19:00") (10:00)	Today 19:00, autoMeridiem false
친구랑 7시 약속 ("Plans with a friend at 7") (20:00) → reply "내일로" ("Make it tomorrow")	Tomorrow 19:00
점심 8천원 썼고 내일 민수 만나. 한 시간 전에 알려줘 ("Spent 8,000 won on lunch, and I'm meeting Minsu tomorrow. Remind me an hour before.")	Expense saved. The event has no time, so it waits on the question "몇 시에 만나요?" ("What time are you meeting?"). The reminder depends on the event time, so it also waits.

4.4 AM / PM Resolution Rules Confirmed

Applies only when a time like "7시" ("7 o'clock") has no cue such as 오전 (AM), 오후 (PM), 아침 (morning), 저녁 (evening), 밤 (night), 새벽 (early morning), or 점심 (lunchtime). If there is a cue, follow it ("저녁 7시" ("7 in the evening") = 19:00).

Date	Rule
Today (or no date mentioned)	Resolve to **the nearest upcoming time after now**. If N AM has not yet passed, use AM; if it has, use PM. If N PM has also passed, do not save and ask "오늘 N시는 이미 지났어요. 내일로 할까요?" ("N o'clock today has already passed. Make it tomorrow?").
Dates from tomorrow onward	1–6 o'clock is automatically treated as PM (13:00–18:00). For 7–11 o'clock, ask "오전이에요, 오후예요?" ("AM or PM?").
12 o'clock	12 noon. If it is today and already past, ask.
"N시 반" ("half past N")	Same rule, plus 30 minutes.
24-hour expressions (13–23 o'clock, 0 o'clock)	Used as-is, no AM/PM resolution. "0시" ("0 o'clock") · "자정" ("midnight") means 24:00 that night (00:00 the next day)

Time of an **expense** (money already spent)	Opposite of events: **the nearest time before now**. E.g. "7시에 밥 7000원" ("meal 7,000 won at 7") at 20:00 → today 19:00. If even N AM today has not yet arrived, do not assume yesterday; ask instead

- When AM/PM was decided automatically, the result card shows **"오후 7시로 저장했어요"** ("Saved as 7 PM") with [수정] ("Edit").
- If the user taps [예] ("Yes") on "내일로 할까요?" ("Make it tomorrow?"), the "dates from tomorrow onward" rule is applied to the same clock number on the next day (for 7–11 o'clock, instead of asking AM/PM again, it is fixed as "PM", because it was postponed after this afternoon's time had passed). Design
- Design The AI returns `meridiemExplicit` (whether there was an AM/PM cue) along with the time, and **the final time is computed in one designated piece of gateway code** according to the table above (not applicable to the app's manual entry form, which uses a time picker). Not leaving this to model guesswork keeps the results deterministic and testable.
- Imprecise times like "저녁쯤" ("around evening") can be stored as an "approximate time" (`FUZZY`). However, if a reminder that requires an exact time was requested, ask for the exact time.

4.5 Grouped Expense Rules Confirmed

Spoken sentence	Saved result
아이스크림 먹었고 생수 샀고 밥 먹었고 총 12,000원 썼어 ("Had ice cream, bought bottled water, had a meal — spent 12,000 won in total")	**1** expense: KRW 12,000, memo "아이스크림, 생수, 밥" ("ice cream, bottled water, meal")
아이스크림 2,000원, 생수 1,000원, 밥 9,000원 썼어 ("Spent 2,000 won on ice cream, 1,000 won on water, 9,000 won on a meal")	Split into **3** expenses
아이스크림 2천원이고 생수랑 밥 해서 총 12000원 ("Ice cream was 2,000 won, and with water and a meal it was 12,000 won in total")	A total is given, so 1 expense (KRW 12,000). Memo "아이스크림 2,000원, 생수, 밥" ("ice cream 2,000 won, bottled water, meal")
아이스크림 2천원 생수 1천원 밥 9천원 총 15000원 ("Ice cream 2,000 won, water 1,000 won, meal 9,000 won, 15,000 won total")	The sum of items (KRW 12,000) differs from the total, so nothing is saved and the app asks "총액이 15,000원이 맞나요?" ("Is the total really 15,000 won?")
아이스크림이랑 생수랑 밥 먹었어 ("Had ice cream, water, and a meal")	No amount, so ask "총 얼마였어요?" ("How much was it in total?")

- Split condition: split into separate expenses **only when an amount was given for every item and no total was given**. If a total is given, it is always 1 expense.
- Category: if all items share the same category, use that category; if mixed, use "기타" ("Other").
- Recognized total expressions: "총", "합쳐서", "다 해서", "전부 해서", "모두" (all meaning "in total / altogether").

4.6 Sending by Voice Confirmed

Example: "내일 오후 7시에 강남역에서 만나기 이걸 민수에게 줘" ("Meet at Gangnam Station tomorrow at 7 PM — give this to Minsu")

1. AI interpretation (1 use deducted): send action, title "만나기" ("Meet"), location "강남역" ("Gangnam Station"), tomorrow 19:00, recipient as spoken "민수" ("Minsu").
2. Recipient matching is done **on the phone** by comparing against friends' aliases and nicknames. The friend list is not sent to the AI.
   - If no name is given ("친구에게 줘" ("give it to a friend")), show the friend list for the user to choose.
   - If multiple friends share the same name, let the user choose. If there is no match, let the user choose from the list.
   - Multiple recipients ("민수랑 지은이한테" ("to Minsu and Jieun")) are also possible.
3. **Send confirmation screen** (same screen as the send sheet): recipient(s), title, location, date · time, reminder, "내 캘린더에도 넣기" ("Also add to my calendar") toggle, [보내기] ("Send").
4. [보내기] ("Send") → sent to server → the sender only sees "전달 대기 중 → 전달 완료" ("Pending delivery → Delivered").

- "내 캘린더에도 넣기" ("Also add to my calendar") is **off** by default. It can be changed to on in Settings > Sending defaults. If the sentence contains "나도 넣고" ("add it for me too"), "내 캘린더에도" ("to my calendar too"), or "같이 넣어줘" ("add it for both of us"), the confirmation screen opens with it on.
- When on, a separate event is created for me, an **independent copy** of the sent item. Editing one does not change the other, and the confirmation screen explains this in one line.
- Sending expenses has the same structure, with a "내 지출에도 넣기" ("Also add to my expenses") toggle (off by default).
- Free members can do this only via the trial (1 AI use + 1 share, 1 recipient, events only). Without a trial remaining, they are directed to the subscription screen.

4.7 AI Usage Deduction Confirmed

- A successful interpretation deducts 1 use. The deduction stands even if the user undoes the saved result.
- Server errors, timeouts, and "cannot interpret" responses **are not deducted from the user's count**. (They are included in the server's cost budget calculation; see 13.8.)
- **Follow-up calls** that re-interpret after the user answers a pending question **are not deducted** (within 10 minutes of the original request, up to 2 per original request). Beyond that, it is treated as a new request and 1 use is deducted. Design
- The server decides what counts as a "non-deducted failure": server error · timeout · failure of server-side validation of the model response · UNSUPPORTED. If the response passes server validation and is returned, but the save is cancelled on the app side or fails app-side validation, it is deducted (the app and server use the same validation rules). Design

5. Friends: Adding · Managing · Profile Photos

5.1 Ways to Add Friends Confirmed

Method	Rule
Invite link	[링크 만들기] ("Create link") → **single-use for 1 person, expires after 5 minutes**. The remaining time is shown, and after expiry [새 링크 만들기] ("Create new link") is offered. When someone with the app installed opens the link while logged in, **they become friends automatically with no acceptance step**. Once connected, the inviter gets a "OO님과 연결됐어요" ("You're now connected with OO") notification and a [연결 끊기] ("Disconnect") button.
QR code	A QR code containing the invite link. Same rules as the link (single-use, 5 minutes).
Nickname search	The nickname must be entered **exactly** for a result to appear (no partial search). A friend request is sent, and they become friends **only after the other person accepts**. Users who turned off searchability do not appear.

- The friend code method is **discontinued** Confirmed . (The code-entry screen in the current design mockup is replaced with Link · QR · Nickname tabs.)
- When someone without the app opens the link, an introduction web page directs them to the store. There is no automatic connection after installation (5-minute expiry); they must get a new link after installing.
- When the link is opened while not logged in: log in → set nickname → (if within 5 minutes) automatic connection. If already expired, show the expiry notice.
- Design Link format: `https://<service domain>/i/<token>`. The token is a random value of at least 128 bits, and only its hash is stored on the server. Opening the app is handled via iOS Universal Links · Android App Links, so `apple-app-site-association` and `assetlinks.json` must be hosted on the domain.

Link-Open Result Screens

Situation	Screen text · behavior
Success	"OO님과 친구가 됐어요" ("You're now friends with OO") → set a display name → on first connection, ask about receive settings (5.3)
Expired (5 minutes elapsed)	"초대 링크가 만료됐어요. 새 링크를 요청해 주세요." ("This invite link has expired. Please ask for a new link.")
Already used	"이미 사용된 링크예요." ("This link has already been used.")
Already friends	"이미 OO님과 친구예요." ("You're already friends with OO.") → navigate to the friend screen
Opening my own link	"내가 만든 링크예요. 친구에게 보내 주세요." ("This is a link you created. Send it to a friend.")
Blocked relationship (either side)	"연결할 수 없는 링크예요." ("This link can't be used to connect.") (the block is not revealed)

Over 30 friends (either side)	"친구는 최대 30명까지예요." ("You can have up to 30 friends.") (if the other side is full: "상대의 친구 수가 가득 찼어요" ("The other person's friend list is full"))
Offline	"인터넷 연결 후 다시 열어 주세요." ("Please connect to the internet and open it again.") The token is not marked as used.

5.2 Managing Friends Confirmed

- Friend limit: **30** per account (same for free and subscribers). Pending outgoing requests also count toward the 30. Design
- **Display name (alias)** Confirmed : for users, like parents, who recognize people by "name" rather than nickname, each friend can be given any **name visible only on my screen** (e.g. nickname "sky_77" → "큰아들 민수" ("eldest son Minsu")).
  - The other person's actual nickname does not change, and they are not notified. It is purely for display on my screen.
  - Where to set it: the "이 사람을 뭐라고 부를까요?" ("What would you like to call this person?") field when a friend first connects (skippable; defaults to the nickname) · [이름 바꾸기] ("Rename") in the friend detail sheet · long-pressing the sender name on a received item card.
  - Rules: 1–20 characters; Korean, English letters, digits, spaces, and emoji allowed; duplicates allowed (may match another friend's name, in which case voice matching shows a choice screen). Clearing it reverts to the nickname.
  - Where it applies: friend list, source label on calendar · expenses ("보낸 사람: 큰아들 민수" ("Sent by: eldest son Minsu")), received item cards, inbox, sent items, arrival notification text, Excel "출처" ("Source") column, voice-send recipient matching. The actual nickname is shown in small text below the name to avoid confusion (friend detail · list).
  - Storage: server `friend_links.alias` (readable only by the owner, used in push notification text) + phone cache. Renaming immediately updates the display on already-received items (items link only to the sender id, and the name is looked up at display time). A deleted user is shown with their last display name + "(탈퇴)" ("(deleted)").
- Friend detail sheet items: profile photo · nickname · alias editing, event receive settings, expense receive settings, notification settings for this friend (arrival · event time), visibility (fully private to this friend / included in public audience), delete all items received from this friend, disconnect, block, report.
- **Disconnect**: two-step confirmation. Exchanged items remain. The other person is not notified. Design
- **Block**: disconnects, and the other person can no longer search for, invite, or send to me. When blocking, show a "이 사람에게서 받은 항목도 삭제" ("Also delete items received from this person") checkbox (off by default). Unblocking is in Settings > Blocked list.
- **Report**: choose a reason (inappropriate profile photo, inappropriate nickname, spam · unwanted sends, other + description) → submitted to the server. Blocking can be done together with reporting. After review, operators can delete the profile photo, reset the nickname, or suspend the account.
- There is **no** in-app chat Confirmed .

5.3 Asking for Receive Settings on First Connection Confirmed

Right after connecting (automatic link connection or right after accepting a request), the receiving side is asked once: "OO님이 일정을 보내면 어떻게 할까요?" ("What should we do when OO sends you an event?") [매번 물어보기(기본)] ("Ask every time (default)") [항상 캘린더에 넣기] ("Always add to calendar") [항상 확인만] ("Always review only"). The same question for expenses is asked on the same screen. Skipping means Ask every time. This screen exists to make it easy to turn on "항상 넣기" ("Always add") for family (parent) connections.

5.4 Profile Photo Confirmed

- Choose from the gallery or take a photo with the camera. Size requirements are handled automatically without being shown to the user.
- Crop screen: **circular guide**, pinch to zoom in · out, drag to reposition, [완료] ("Done").
- Storage: downscaled on the device to a **512×512 square JPEG (quality about 0.8)**, **EXIF (location data, etc.) stripped**, then uploaded. Cropped to a circle for display.
- Default avatar: the initial (first character) of the nickname (or alias).
- Visibility: visible **to friends only**. Nickname search results and the link introduction page show only the initial.
- Can be changed or deleted. Deleted on account deletion. Photos of blocked users are hidden.
- Storage location: private bucket `avatars/{user_id}/{version}.jpg`, displayed only via expiring signed URLs. When a new version is uploaded, the previous file is deleted. Design

6. Sharing: Sending and Receiving Events and Expenses

6.1 Core Principles Confirmed

- Sending is a **subscriber feature** (free members get 1 sharing trial). Free members can also receive.
- Free members can only view each other's public events; they cannot send (except for the 1 sharing trial).
- Delivery is **immediate only** (no scheduled delivery).
- The sender **cannot know** whether the recipient "added it / only viewed it". The sender sees only "Pending delivery / Delivered / Expired / Canceled".
- The recipient never sees states such as "pending delivery" or scheduling. They get a notification the moment the item arrives.
- Received items always show **who added them** (e.g., "보낸 사람: 민수" (Sent by: Minsu)).
- The recipient can freely delete received items at any time, and can delete **all items sent by one person at once**.
- The sender's **edits and cancellations take effect immediately**. If the sender edits again, it overwrites even changes the recipient made themselves. If the recipient does not want this, they can switch that friend's receiving setting to "Do not receive".
- Items deleted by the recipient are **never restored** (even if the sender edits them later).

6.2 Send Screen (Send Sheet) Confirmed

Item	Details
Recipients	Multi-select friend chips. Friends set to "Do not receive" are disabled and show "이 친구는 받기를 꺼 두었어요" (This friend has turned off receiving). Design
Event fields	Title (required), date (required), time (exact time or all-day), end time (optional), location (optional)
Expense fields	Amount (required), category, memo, date and time
Reminders (event)	Multi-select from: at start time, 10 min before, 30 min before, 1 hour before, 1 day before. If none is selected, the recipient's default reminder setting applies. Design
Also add to my calendar	Toggle (off by default; default can be changed in Settings). For expenses: "Also add to my expenses".
Remaining count	Free members see "공유 체험 1회 남음" (1 sharing trial left). Subscribers see "오늘 보낼 수 있는 건수 N건" (N sends left today) only when usage is at 80% or more of the limit. Design
[Send] result	Per recipient: "보냈어요" (Sent) or the failure reason. Subsequent status is shown in the "Sent items" list.

6.3 Recipient-Side Handling: Receive Modes Confirmed

The recipient sets a receive mode **per friend and per type (event / expense)**.

Receive mode	Behavior
Ask every time (default)	Arrival notification → received card: [Add to calendar] [View only] + checkbox "OO님 일정은 항상 이렇게" (Always do this for OO's events). If checked when tapping, that friend's mode changes to "Always add" or "Always view only".
Always add	The app automatically adds the item to the calendar (or expenses) as soon as it receives it and schedules reminders. Because the server sends the arrival notification at send time, it is worded "OO님이 일정을 보냈어요(캘린더에 자동으로 들어가요)" (OO sent you an event (it will be added to your calendar automatically)).
Always view only	Placed in the inbox only. Arrival notification: "OO님이 일정을 보냈어요" (OO sent you an event).
Do not receive	Sends of that type from that friend are not received. Shown as disabled on the sender's send screen.

- **Preventing abuse of Always add**: If one sender sends more than 20 items to the same recipient in a day (KST), from the 21st onward items are handled as "View only" for the rest of that day, and the recipient gets one notification: "OO님이 오늘 많은 일정을 보냈어요. 수신함에서 확인하세요" (OO sent you many events today. Check your inbox).
- "View only" items are kept in the inbox for **30 days**, during which [Add to calendar] can be tapped. After 30 days they are automatically deleted from the inbox.
- Received expenses are **included in my expense total** by default. A per-item toggle and a per-friend default ("Include expenses sent by this friend in the total") are provided.
- Events whose start time has already passed cannot be sent ("이미 지난 일정은 보낼 수 없어요" (You can't send an event that has already passed)). If an item fetched just before expiry is saved in the app after its start time has passed, it is added to the calendar but no reminder is scheduled, and its status is set to "Past". Design

6.4 Delivery Status (What the Sender Sees) Confirmed

Status	Condition
Pending delivery	Saved on the server but not yet fetched by the recipient's app. This state can last a long time if the recipient's phone is off or the app is not run.
Delivered	The recipient's app fetched the item, saved it on the phone, and sent an acknowledgment (ack).
Expired	While pending delivery, **the event start time passed or 30 days elapsed** (30 days for expenses). For all-day and approximate-time events, 23:59 (KST) that day is treated as the start time. Once expired, the item is no longer delivered to the recipient. Deliveries ended by the recipient blocking, switching to Do not receive, or deleting before delivery also appear to the sender as "Expired".
Canceled	The sender canceled it.

The sender is not warned even if notifications are turned off on the other person's device Confirmed .

6.5 Edit, Cancel, Delete Confirmed

- **Sender edits**: Edit from "Sent items" → server version +1 → overwrites the copy on the recipient's phone → received item shows **"수정됨 · 10/2 15:30" (Edited · 10/2 15:30)** (a prominent tag) → reminders rescheduled → recipient gets the notification "OO님이 일정을 수정했어요" (OO edited an event) (follows the arrival notification setting).
- Edits are also applied to items sitting in the inbox as "View only".
- **Sender cancels**: The event on the recipient's phone changes to status "Canceled", shows "보낸 사람이 취소했어요" (The sender canceled this), and its reminders are canceled. Expenses are deleted from the recipient's phone (excluded from totals). If not yet delivered, it is not delivered.
- **Recipient deletes**: Deleted from the phone and a deletion record is sent to the server. Subsequent sender edits or cancellations are not applied for that recipient. The sender cannot know about the deletion (it stays "Delivered").
- **Delete everything from one person at once**: Friend details > "이 친구에게서 받은 항목 모두 삭제" (Delete all items received from this friend) → two-step confirmation → all those items deleted + deletion records.
- If the sender deletes their account, received items remain and the sender label changes to "탈퇴한 사용자" (Deleted user). No further edits or cancellations arrive.
- A sender whose subscription has expired is blocked only from sending new items; they can still edit and cancel items already sent `Confirmed` .

6.6 Limits `Confirmed`

Limit	Value
Subscriber daily sends	50 (reset at KST midnight). Counted as 1 per recipient (sending to 3 people counts as 3).
Always add automatic handling	20 per day per sender per recipient; excess is View only
Free member sharing trial	Once per lifetime, events only, 1 recipient
Pending delivery retention	Until the event start time or 30 days, whichever comes first
Inbox ("View only") retention	30 days

6.7 End-to-End Flow (Sending → Receiving an Event)



> **Important technical limitation**: The server cannot write directly to the local DB on the recipient's phone. A received event is actually added to the calendar and its local reminders scheduled only **when the recipient's app runs a sync**. Even when a push arrives, iOS does not always run app code in the background. To compensate, we use the **server fallback notification** in 13.7: when the reminder time of an item still pending delivery arrives, the server sends the event reminder via remote push instead. This must be tested on real devices (iOS and Android, including battery saver mode).

7. Visibility · Viewing Friends' Events

7.1 Visibility Rules Confirmed

- Friends can see only **events and tasks**. **Expenses are never visible to friends.**
- Each event and task is set to [Public / Private]. The default for new items is chosen by the user at first login and can be changed in Settings.
- Per-friend settings:
  - "Hide everything from this friend": nothing is visible to that friend.
  - Audience mode: [Public to all friends (default)] or [Public to selected friends only]. If the latter is chosen, each friend gets an "audience" checkbox.
- **The most restrictive setting wins.** Conditions for friend F to see item X:

```
X.visibility = PUBLIC
AND F is not on the "hide everything" list
AND (audience mode = all friends OR F is included in the audience)
AND the friendship is valid on both sides (not blocked or disconnected)
```

- Public events show **title and time (including location)** Confirmed . Memo, reminders, and source are not shown. Design
- Events received from others are private by default (so sender information is not exposed to third parties). Design

7.2 Friend Events Screen

- People tab > select friend > [View events] → monthly calendar of that friend's public events (read-only).
- Turning on "Overlay friends' events" in my calendar shows the public events of selected friends (up to 3) as dots of different colors, and marks "Overlap" when their times overlap with my events. Design
- Tapping a friend's event allows [Copy to my calendar] (no subscription needed; source "OO님 일정에서 복사" (Copied from OO's event)). Design

7.3 Server-Side Approach Design

Backups are readable only by their owner. Events and tasks to show to friends are uploaded to a separate table `public_events` (kind = EVENT | TASK) — **public items only**, with only the needed fields (title, start/end, all-day flag, approximate time, location / for tasks: due date and completion status, updated time). The friend events screen shows tasks as a list under each date. When an item is made private or deleted, it is removed from this table immediately. Friend lookups are allowed only through a DB function that checks the 7.1 conditions.

8. Notifications

8.1 Notification Types

Type	Sent by	Description
Event time reminder	Phone (locally scheduled)	Reminders for my events and received events. Fire without the server. Received events not yet delivered use the server fallback notification (13.7).
Arrival notification	Server (remote push)	A friend sent / edited / canceled an event or expense
Friend notification	Server	Friend request received, connected via link
System notification	Server	Login on another device, upcoming subscription expiry (3 days before, optional)

8.2 Notification Settings Structure Confirmed

Level	Items
Global	All notifications on / off
Me	My event reminders on / off, **default reminder** (none / at start time / 10 min before / 30 min before / 1 hour before; default "at start time")
Per friend	Arrival notifications on / off, event time reminders on / off (for events sent by that friend)
Display	Show event title on the lock screen (on by default). When off, shown without a title, e.g., "새 일정이 도착했어요" (A new event has arrived).

- Precedence: **Global off > per-friend settings > per-item reminders**. The more restrictive setting applies.
- A reminder stated explicitly in the utterance ("1시간 전에 알려줘" (Remind me 1 hour before)) takes precedence over the default reminder.
- Arrival notification settings and lock screen display settings are also stored on the server, since the server needs them (the server composes the push text).
- If notification permission is missing, the reminder state is saved as `PERMISSION_BLOCKED` and the event card shows "알림 꺼짐" (Notifications off). A reminder is shown as scheduled only after scheduling succeeds (`SCHEDULED` does not guarantee delivery).
- Reminders are rescheduled on restore, device change, and edits to received events. If the event time changes, they are recalculated using the relative offset.
- Verify Limit on the number of scheduled local notifications (reportedly 64 on iOS). Handle by scheduling the nearest upcoming ones first and refilling on app launch. Android exact alarm permission (`SCHEDULE_EXACT_ALARM` / `USE_EXACT_ALARM`) policy and Android 13+ notification permission (`POST_NOTIFICATIONS`).

8.3 Notification Text Design

Situation	Title display on	Title display off
Arrived via Always add	어머니가 일정을 보냈어요(캘린더에 자동으로 들어가요) · 병원 예약 10/3(금) 오후 3시 (Mom sent you an event (it will be added to your calendar automatically) · Hospital appointment 10/3 (Fri) 3 PM)	어머니가 일정을 보냈어요 (Mom sent you an event)
Arrived via Ask every time	민수가 일정을 보냈어요 · 만나기 10/3 오후 7시 (Minsu sent you an event · Meetup 10/3 7 PM)	민수가 일정을 보냈어요 (Minsu sent you an event)
Expense arrived	아들이 지출을 보냈어요 · 병원비 30,000원 (Son sent you an expense · Hospital bill KRW 30,000)	아들이 지출을 보냈어요 (Son sent you an expense)
Edit	민수가 일정을 수정했어요 · 만나기 → 오후 8시 (Minsu edited an event · Meetup → 8 PM)	민수가 일정을 수정했어요 (Minsu edited an event)
Cancel	민수가 일정을 취소했어요 · 만나기 (Minsu canceled an event · Meetup)	민수가 일정을 취소했어요 (Minsu canceled an event)
Event time	병원 예약 · 30분 후 (강남세브란스) (Hospital appointment · in 30 min (Gangnam Severance))	일정 알림 · 30분 후 (Event reminder · in 30 min)

Names prefer the alias the recipient assigned. The server knows the alias (stored in friend settings), so it uses it in the push text.

9. Subscriptions · Trials · Limits · Payments · Costs

9.1 Plans Confirmed

Plan	Price (monthly, VAT included)	AI parsing	Sending
Free member	KRW 0	3 trials (lifetime)	1 trial (lifetime)
Basic	KRW 2,900	150 per month, 20 per day	50 per day
Plus	KRW 7,900	500 per month, 40 per day	50 per day

- Monthly limits reset each **subscription billing cycle** (1 month from purchase date); daily limits reset at **KST midnight**. Design
- When upgrading from Basic to Plus, usage in the current cycle carries over and only the limit increases (e.g., 120 used → 380 remaining). If the upgrade starts a new store billing cycle, the server moves the usage from the previous monthly bucket to the new bucket. The monthly bucket key is the cycle start time reported by the store (`period_start`). Design
- Trials are per account for life; while subscribed, trial uses are not consumed and are preserved. Any remaining trials can be used again after the subscription expires. Design
- Store-level "free trial period" products (e.g., 7 days free) are not included in phase 1.
- Annual billing products are not included in phase 1. Design (Not blocked structurally, since adding one only requires creating the product)

9.2 Subscription Screen (Paywall)

- Entry points: trials exhausted, limit exceeded, attempt to send (free), Settings > Subscription.
- Contents: two plan cards (price, limit numbers, included features), current plan indicator, [Subscribe], **[Restore purchases]**, auto-renewal notice text, Terms of Service and Privacy Policy links, instructions for canceling.
- Verify Apple and Google required disclosure text for auto-renewing subscriptions and where it must be displayed.

9.3 When a Subscription Expires Confirmed

- New sends, expense sends, and AI use (beyond trials) are blocked.
- Editing and canceling already-sent items, received items, friends, receive settings, and backups remain unchanged.
- Subscription benefits are kept during the store grace period caused by a payment failure. Design

9.4 Payment Processing Confirmed

- As a digital subscription, **only store in-app purchases** are used. No external payment links are placed in the app.

- After purchase, the app sends the receipt (Google: purchaseToken, Apple: signed transaction info) to the server, and subscription entitlement is granted only based on **the server's verification with the store**. Features are never unlocked based solely on the app's judgment.
- Renewals, cancellations, refunds, and payment failures are received and applied by the server via store server notifications (App Store Server Notifications v2, Google Real-time Developer Notifications).
- [Restore purchases] re-verifies the store purchase history with the server and attaches the entitlement to the current account. If the purchase is already attached to another account, show "다른 계정에서 사용 중인 구독이에요" (This subscription is in use on another account). `Design`

9.5 Cost Basis

Item	Value
AI unit price (gpt-4.1-mini)	Input $0.40 / 1M tokens, output $1.60 / 1M tokens (OpenAI official docs, checked 2026-10-02)
Per-call assumption	Input about 1,500 tokens (including instructions and schema), output about 200 tokens → about $0.0009 ≈ **KRW 1.3** (assuming an exchange rate of KRW 1,400/$)
Conservative unit price used for pricing	KRW 5 per call
Basic plan net revenue estimate	KRW 2,900 ÷ 1.1 (VAT) × 0.85 (assuming 15% fee) ≈ KRW 2,240. Even if the full limit is used (150 calls × KRW 5 = KRW 750), about KRW 1,490 remains
Plus plan net revenue estimate	KRW 7,900 → about KRW 6,100. 500 calls × KRW 5 = KRW 2,500 → about KRW 3,600 remains
Fixed cost estimate	Supabase Pro about $25/month + Apple Developer $99/year → about KRW 47,000 per month. Fixed costs are covered from about 30 subscribers

`Verify` Store fees (assuming 15% under the small business program; 30% if not enrolled), VAT handling, exchange rate, Supabase pricing, actual token usage. After launch, limits can be raised based on measured usage (server configuration values).

10. Settings · Font Size · Excel Export

10.1 Settings Screen Layout

Section	Items
My profile	Profile photo, nickname (next date it can be changed), allow nickname search
Subscription	Current plan, AI usage this month / today, remaining trials, [View plans], [Restore purchases], go to store subscription management
Notifications	Global, Me (default reminder), per friend, show title on lock screen, notification permission status · [Open settings]
Visibility	Default visibility for new events, audience mode (all friends / selected friends), per-friend hide-everything list
Sending	Default for "보낼 때 내 캘린더(지출)에도 넣기" (Also add to my calendar (expenses) when sending)
Friends · Blocking	Receive settings summary, block list · unblock
Display	Font size, (existing) display settings
My data	Backup status (last backup time, pending count), export to Excel, reset (a)(b)(c)
Account	Login method shown, log out, delete account
About	Terms of Service, Privacy Policy, contact us, open source licenses, app version

10.2 Font Size Confirmed

- Range **70% to 200%**, default 100%, **14 steps in 10% increments** (70, 80, …, 200).
- Controls: drag the slider left/right, or use the **[−] [+] buttons** on either side to move one step. Shows the current scale number ("130%"). A preview sentence above. A [Reset to default] button.
- "Follow system font size" switch: when on, follows the device setting and the slider is disabled. When off, only the app scale applies (not multiplied by the system scale).
- Implementation: all text sizes go through a single scale function (`scaleFont(px)`). Define layout rules, such as spacing and icons reflecting only half of the scale. Design
- Done criteria: **at 200%, every screen is usable without clipping**, minimum touch target of 44px is maintained, and numeric columns may wrap.

10.3 Excel Export Confirmed

- Format `.xlsx`, 2 sheets: "지출" (Expenses), "일정" (Events). (`.csv` is not used due to concerns about Korean text becoming garbled)
- Options: period (all / this month / custom), type (expenses / events / both).
- Expense columns: date, time, type (expense/income), amount (number format #,##0), category, memo, input method (manual/text/voice), source ("내가 입력" (Entered by me) / "OO님이 입력" (Entered by OO)), included

in total (yes/no).

- Event columns: start date, start time (if approximate time, that expression), end, title, location, status (upcoming/past/done/canceled), reminders (list of minutes before), source, visibility (public/private), edited (yes/no).
- Deleted items are excluded. Received items are included and distinguished by "Source".
- Times are displayed in the device time zone.
- **Security**: To prevent text starting with `= + - @` from being executed as formulas, all text cells are forced to string format.
- Generated on the phone (no server involved; guests can use it too) → saved or sent via the phone's share feature. Example filename: `VoiceLife_2026-10-02.xlsx`.
- Implementation requires adding an Excel generation library and a file sharing module (e.g., a SheetJS-family library or exceljs, `expo-file-system`, `expo-sharing`). The library choice will be made during frontend implementation.

11. Screen Design (Frontend Screen List and Flow)

11.1 Navigation Structure Design

4 bottom tabs + settings button in the top right: **Home · Calendar · Expenses · People**. Sheets (screens that slide up from the bottom) are used for Send · Add friend · Friend details · Received item · Profile crop. Inherits the Home · Settings · Plans · People pages and sheet structure from the current design mockups (`design/`).

11.2 Screen List

ID	Screen	Entry	Key elements · states
O1	Welcome	First launch	App introduction, [Get started]
O2	Notification permission explanation	After O1	Explanation text, [Continue] (system popup), [Later]
O3	Login choice	After O2 / features requiring login	Apple · Google · Kakao buttons, [Start without logging in], terms and age 14+ consent, help on switching platforms
O4	Nickname setup	First login	Input field, real-time rule and duplicate check results, [Next]
O5	Default visibility setup	After O4	[Public] [Private] explanation cards
O6	Backup choice	When both a backup and guest records exist	[Load server backup] [Overwrite with this phone's records], counts for both
O7	Restoring	Backup restore	Progress, [Retry] on failure, permission guidance after completion
H1	Home	Tab	Quick entry (mic · input field), AI usage display (subscription · trial), backup-off banner (guest), today's events, recent expenses, received items badge
H2	Result card	After input processing	Per-action cards (saved / question / failed), [Undo], [Edit], auto-decision indicator ("오후 7시로 저장했어요" (Saved as 7 PM)), question input
H3	AI required notice	Utterance that needs AI	Guest: login prompt / Free: confirm trial use or subscription prompt / limit exceeded notice, [Enter manually]
H4	Manual entry form	H1, H3	Expense tab / Event tab, prefilled with the spoken sentence
K1	Calendar	Tab	Month · week view, event dots, received event indicator (sender avatar), friend overlay toggle, [+]
K2	Event details	K1	Title, time, location, reminders, visibility toggle, source · edited · canceled indicators, [Send], [Edit], [Delete]
E1	Expenses	Tab	Monthly total (reflects whether received expenses are included), by category, list (with source)

E2	Expense details	E1	Amount, category, memo, include-in-total toggle (received items), [Send], [Edit], [Delete]
P1	People	Tab	Friend list (photo · alias · nickname), received friend requests, inbox shortcut, sent items shortcut, [Add friend]
P2	Add friend sheet	P1	Tabs: Link (create · share · time remaining · new link) / QR (show · scan) / Nickname (exact search · send request)
P3	Link result	Opening a link	Per-result text from 5.1
P4	Display name · receive settings	First connection	"이 사람을 뭐라고 부를까요?" (What would you like to call this person?) input; for events and expenses each: [Ask every time] [Always add] [Always view only]
P5	Friend details sheet	P1	All items from 5.2; dangerous actions require two-step confirmation
P6	Friend events view	P5	Read-only monthly calendar, [Copy to my calendar]
S1	Send sheet	K2, E2, P5, voice send	All items from 6.2
S2	Sent items	P1	List, per-recipient status (pending delivery / delivered / expired / canceled), [Edit], [Cancel]
R1	Received item card	Arrival notification, P1 inbox	Sender, content, [Add to calendar] [View only], "항상 이렇게" (Always do this) checkbox
R2	Inbox	P1	"View only" items for 30 days, days remaining, [Add], [Delete]
B1	Subscription (paywall)	9.2	Items in 9.2
T1	Settings	Top button	Sections in 10.1
T2	Profile · crop	T1	Photo selection, circular crop (pinch · drag)
T3	Notification settings	T1	Structure in 8.2
T4	Visibility settings	T1	Settings in 7.1
T5	Font size	T1	10.2
T6	My data	T1	Backup status, Excel export, reset (a)(b)(c)
T7	Account · delete account	T1	Log out (un-backed-up data warning), two-step account deletion, subscription cancellation guidance
T8	Block list · report	T1, P5	Unblock, choose report reason

11.3 Common Screen State Rules

- Every list screen has designs for **empty · loading · error (retry) · offline** states.

- Buttons that require the server are disabled when offline + "인터넷 연결이 필요해요" (An internet connection is required). Exception: **Send** can be tapped offline; it is queued in the outbox and sent when connected (4.2 · Chapter 15).
- Irreversible actions use a red (danger) button + two-step confirmation.
- After an automatic save, [Undo] is shown at the bottom of the screen for a few seconds.
- Received items always show the sender's avatar and name; edited items get an "Edited" tag, and canceled items get a strikethrough + "Canceled".

11.4 Changes from the Current Design Mockups (`design/`)

Current mockup	Change
4 buttons for received events (accept this time · decline this time · always accept · always decline)	[Add to calendar] [View only] + "Always do this" checkbox
Send results "added right away / awaiting acceptance / declined"	After "Sent", statuses are only pending delivery / delivered / expired / canceled
Copy · enter friend code	Link (5 min · single use) / QR / Nickname tabs
Friend request acceptance screen upon receiving a link	Auto-connection result screen + receive settings question
One kind of receive setting	Separate for events and expenses, add "Do not receive"
No expense send screen	Add expense send sheet
Simple notification settings	Global · Me · per friend + arrival / event time + default reminder + lock screen
No source · edited · visibility indicators	Add them
"Unlimited" plan example	Basic KRW 2,900 / Plus KRW 7,900 with numeric limits
None	Login · nickname · backup choice · restore, inbox, sent items, profile crop, font size, my data (Excel · reset), block · report, paywall, "Also add to my calendar" toggle

12. Frontend Implementation Design

12.1 Layer Structure

```
UI / Features (screens, components)
   ↓
Application (input router, proposal generation, validators, executors, sync engine, limit display)
   ↓
Domain (types and rules for Event, Transaction, Task, Reminder, SharedCopy, Friend, etc.)
   ↓
Repository interfaces
   ↑ implements
Adapters (SQLite storage, Supabase client, AI gateway client, speech recognition, notifications, pa
yments, file sharing)
```

- AI never changes the local DB directly. Every change follows the order **proposal** → **validation** → **execution** → **repository** → **SQLite** → **ActionLog**. Applying received items and restores also go through the repository layer (recorded in ActionLog with source "sync").
- Platform-dependent code (notifications, payments, speech, files) sits behind adapters.
- Folders (proposed): `src/domain/{event,transaction,task,reminder,sharing,friend}`, `src/application/{proposals,executors,sync,backup,sharing,entitlement}`, `src/adapters/{sqlite,supabase,ai,notifications,iap,speech,export}`, `src/features/{home,calendar,expense,people,send,inbox,settings,onboarding,paywall}`.

12.2 Gaps from the Current Code (To Implement) — Repository Review Results

Current state found	To do
Only an expense save executor exists (no event executor)	Event create · update · delete executors, reminder executor, task executor
No Task domain file (Reminder already has a TASK target)	Define the Task domain: title, due date (optional), completed flag, visibility, source. In phase 1 scope, visibility and reminder rules are the same as for events
Event · Transaction lack sender and source fields	Migration in 12.3
Local parser handles single expenses only	Keep (free tier). Bundles, compound inputs, and events go through the AI path
AI gateway contract returns only a single expense proposal	Extend to the multi-action contract in 14.4 (jointly with backend)
AI connection uses a manual server address + pilot token	Automatic authentication with the Supabase login token; hide the connection settings screen as developer-only
Design mockups reflect the previous spec	Apply 11.4

12.3 Local DB Changes (Migrations) Design

All schema changes are made via migration files. Added columns give existing data a default value.

```
Columns added to events / transactions / tasks
  origin              TEXT  NOT NULL DEFAULT 'LOCAL'   -- LOCAL | RECEIVED | COPIED
  shared_item_id      TEXT  NULL      -- server shared_items.id if this is a received item
  delivery_id         TEXT  NULL
  sender_user_id      TEXT  NULL
  sender_name         TEXT  NULL      -- snapshot of the display name at receipt
  sender_state        TEXT  NULL      -- ACTIVE | DISCONNECTED | DELETED
  remote_version      INTEGER NULL
  sender_edited_at    TEXT  NULL      -- for the "Edited" indicator
  sent_copy_of        TEXT  NULL      -- if this is a copy made via "Also add to my calendar", the
sent item id (no linkage, display only)
Added to events / tasks
  visibility          TEXT  NOT NULL DEFAULT 'PRIVATE'   -- PUBLIC | PRIVATE. The actual default is
filled in by the app from settings on save. Guest records are always PRIVATE
Added to transactions
  include_in_total    INTEGER NOT NULL DEFAULT 1
  bundle_items        TEXT  NULL      -- bundled expense items JSON (preserved separately from mem
o, optional)

New tables
  friends_cache(user_id PK, nickname, alias, avatar_version, receive_mode_event, receive_mode_expen
se,
                notify_arrival, notify_time, hidden_all, in_selected_audience, include_expense_in_t
otal, state, updated_at)
  inbox_items(delivery_id PK, shared_item_id, kind, payload_json, sender_user_id, received_at, expi
res_at, version)
  sent_items_cache(shared_item_id PK, kind, payload_json, created_at, updated_at, version, status)
  sent_deliveries_cache(delivery_id PK, shared_item_id, recipient_user_id, state, updated_at)
  received_tombstones(shared_item_id PK, deleted_at)    -- prevents resurrection (also recorded on
the server)
  outbox(id PK, kind, entity_type, entity_id, payload_json, created_at, attempts, last_error, stat
e)
  sync_state(key PK, value)     -- sync cursor, last backup time, active device id
  settings(key PK, value)       -- font scale, notification settings, send defaults, visibility def
aults, etc.
  pending_drafts(id PK, text, created_at)   -- drafts awaiting AI while offline (excluded from back
up)
```

12.4 Sync Engine Design

- **Upload (backup · send · deletion records)**: Process the outbox in order. Attach a `clientRequestId` (UUID) to each request so the server ignores duplicates (idempotent). On failure, exponential backoff (1s, 2s, 4s … up to 5 min); after 5 or more failures, show an error in Settings > My data.
- **Download (receiving · friend changes · delivery status)**: Call `GET sync?cursor=` on app start, switch to foreground, push receipt, and pull-to-refresh. Store the cursor from the response.
- Order for handling received deliveries: check deletion records (if present, ignore and ack) → handle according to the server-determined `routed_mode` (ALWAYS_ADD: save to calendar/expenses / ASK: received card / ALWAYS_VIEW: inbox) → schedule reminders → ack.
- If a higher version of the same `shared_item_id` arrives, overwrite. Lower or equal versions are ignored.
- Backup upload is a per-entity upsert (events · expenses · tasks · reminders · settings · received copies · inbox · deletion records). `rawInput` is not sent.

- Public event sync: creating, editing, or making an event public upserts into `public_events`; making it private or deleting it deletes from there.

12.5 Status · Entitlement Display

- Subscription, usage, and trials are cached from the server's `GET entitlement` for display only. The server decides what is actually allowed.
- Undo: right after an automatic save, revert using the inverse-action info in ActionLog. Sends already sent to the server are handled as "Cancel", not undo.

12.6 Design Tokens · Accessibility

- Colors: ink `#172B40`, primary `#194F85`, muted · canvas · line · paper · danger (keep the existing `expense-styles.ts` palette).
- Spacing, radius, and font size are managed as tokens, and font sizes go through `scaleFont`.
- Every button has an accessibility label (VoiceOver · TalkBack). Minimum touch target 44px. State is never conveyed by color alone; text and icons are used alongside.

13. Backend Implementation Design (Supabase)

> The schema below is **design-level DDL** intended to make the intent clear. The backend owner may adjust names and detailed types, but **the meaning of the constraints (UNIQUE, CHECK, FK) and access rules (RLS) must be preserved**. RLS is enabled on every table.

13.1 Accounts · Profiles · Devices

```
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  nickname citext not null unique check (char_length(nickname) between 2 and 12
                                      and nickname ~ '^[가-힣A-Za-z0-9]+$'),
  nickname_changed_at timestamptz not null default now(),
  searchable boolean not null default true,
  default_visibility text not null check (default_visibility in ('PUBLIC','PRIVATE')),
  audience_mode text not null default 'ALL_FRIENDS' check (audience_mode in ('ALL_FRIENDS','SELECTE
D')),
  avatar_version int,                    -- null means show initials
  default_reminder_offset int,           -- default reminder (minutes, 0 = at start time, null = no
ne). Default 0. Used for server fallback reminders
  lockscreen_show_title boolean not null default true,
  notify_all boolean not null default true,
  age_confirmed_at timestamptz not null,
  terms_version text not null,
  status text not null default 'ACTIVE' check (status in ('ACTIVE','SUSPENDED')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
create table nickname_holds (nickname citext primary key, available_at timestamptz not null);
create table active_devices (
  user_id uuid primary key references profiles(id) on delete cascade,
  device_id text not null, platform text not null check (platform in ('ios','android')),
  push_token text, session_id uuid not null, updated_at timestamptz not null default now()
);
```

- **One account, one device**: right after login the app calls `register_device` → `active_devices` is replaced with the new device → the previous device's session is invalidated. Every protected function returns `409 DEVICE_REPLACED` if the request's `device_id` differs from the active device. **Verify** How to force-terminate other sessions in Supabase Auth (admin API).
- The nickname change function handles the 30-day limit, banned-word check, `nickname_holds` check, and registering a 30-day hold on the previous nickname, all in one transaction.

13.2 Friends · Invites · Requests · Blocks · Reports

```
-- One row per direction (friend settings as seen by me). On connect, both rows are created in the
same transaction.
create table friend_links (
  owner_id uuid references profiles(id) on delete cascade,
  friend_id uuid references profiles(id) on delete cascade,
  state text not null check (state in ('ACTIVE','DISCONNECTED')),
  alias text check (alias is null or char_length(alias) between 1 and 20),   -- display name on my
screen
  receive_mode_event text not null default 'ASK'
     check (receive_mode_event in ('ASK','ALWAYS_ADD','ALWAYS_VIEW','NOT_ALLOWED')),
  receive_mode_expense text not null default 'ASK' check (...same values...),
  notify_arrival boolean not null default true,
  notify_time boolean not null default true,
  hidden_all boolean not null default false,        -- everything private to this friend
  in_audience boolean not null default false,       -- target of selective sharing
  include_expense_in_total boolean not null default true,
  connected_at timestamptz, updated_at timestamptz not null default now(),
  primary key (owner_id, friend_id), check (owner_id <> friend_id)
);
create table invite_links (
  id uuid primary key, inviter_id uuid not null references profiles(id) on delete cascade,
  token_hash bytea not null unique, expires_at timestamptz not null,   -- creation + 5 minutes
  used_by uuid, used_at timestamptz, created_at timestamptz not null default now()
);
create table friend_requests (
  id uuid primary key, from_id uuid not null, to_id uuid not null,
  state text not null check (state in ('PENDING','ACCEPTED','DECLINED','CANCELLED')),
  created_at timestamptz not null default now(),
  unique (from_id, to_id) where (state = 'PENDING')   -- implemented as a partial unique index
);
create table blocks (blocker_id uuid, blocked_id uuid, created_at timestamptz default now(),
                     primary key (blocker_id, blocked_id));
create table reports (id uuid primary key, reporter_id uuid, target_id uuid,
  reason text check (reason in ('AVATAR','NICKNAME','SPAM','OTHER')), detail text,
  state text default 'OPEN' check (state in ('OPEN','ACTIONED','DISMISSED')),
  created_at timestamptz default now(), handled_at timestamptz);
```

- Redeeming a link (`redeem_invite`) is a single transaction: lock the row by token hash (`for update`) → check expiry · already used · self · blocked · already friends · 30-friend limit → record `used_by` → create the two `friend_links` rows (if a DISCONNECTED row already exists, restore it to ACTIVE so previous items carry over) → push to the inviter. Even if two people open it simultaneously, only one succeeds.
- The nickname search function returns exactly one user only on an exact match + `searchable = true` + no block relationship, and the result does not include a photo. Search is rate-limited to 10 per minute per user (to prevent brute-force lookups).
- The 30-friend cap is checked inside the connect function. Only ACTIVE connections + PENDING requests I sent are counted (DISCONNECTED excluded). To prevent concurrent connections from reaching 31, both users' `profiles` rows are locked with `for update` in id order before checking.
- Connection creation is an upsert. If a previous DISCONNECTED row exists, it is restored to ACTIVE and the existing settings (alias · receive mode) are kept. If A→B and B→A requests are both pending at the same time, they are connected automatically.

13.3 Shared Items · Deliveries · Deletion Records

```
create table shared_items (
  id uuid primary key, sender_id uuid not null references profiles(id) on delete cascade,
  kind text not null check (kind in ('EVENT','EXPENSE')),
  payload jsonb not null,          -- 13.4 schema, validated on the server
  version int not null default 1,
  status text not null default 'ACTIVE' check (status in ('ACTIVE','CANCELLED')),
  starts_at timestamptz,           -- EVENT start time (ALL_DAY·FUZZY use 23:59 KST that day), for
expiry calculation
  client_request_id uuid not null, -- idempotency
  created_at timestamptz not null default now(), updated_at timestamptz not null default now(),
  unique (sender_id, client_request_id)
);
create table deliveries (
  id uuid primary key, item_id uuid not null references shared_items(id) on delete cascade,
  recipient_id uuid not null references profiles(id) on delete cascade,
  state text not null default 'PENDING' check (state in ('PENDING','DELIVERED','EXPIRED','CANCELLE
D','WITHDRAWN')),
                                -- WITHDRAWN: ended by block · "do not receive" · deletion before
delivery (shown to sender as 'Expired')
  routed_mode text not null check (routed_mode in ('ASK','ALWAYS_ADD','ALWAYS_VIEW')),  -- decided
at send time
  expires_at timestamptz not null, delivered_at timestamptz, delivered_version int,
  recipient_deleted_at timestamptz,          -- must never be exposed to the sender
  is_trial boolean not null default false,
  created_at timestamptz not null default now(),
  unique (item_id, recipient_id)
);
```

- **`deliveries` is never exposed directly to the sender.** The RLS SELECT policy only allows `recipient_id = auth.uid()`. The sender-facing view (or function) `sent_delivery_status` returns only `id, item_id, recipient_id, state, delivered_at`. `recipient_deleted_at` and `routed_mode` are visible only to the recipient. This is the server-side implementation of the rule "the sender does not know whether the item was accepted."
- `routed_mode`: at send time the server determines the recipient's receive mode and records it, and **this value is final**. The app processes the item according to routed_mode. If the mode is ALWAYS_ADD but that sender already has 20 items routed as ALWAYS_ADD to that recipient today (KST), it is recorded as ALWAYS_VIEW. The counter is incremented atomically with `insert … on conflict … do update set n = n + 1 returning n`, and only counts when the result is ALWAYS_ADD.
- If, after sending, the recipient switches that friend to "do not receive" or blocks them, any deliveries still PENDING end as WITHDRAWN.
- `starts_at` is always populated: for EXACT, the start time; for ALL_DAY · FUZZY, 23:59 KST that day. `expires_at` = `least(starts_at, created_at + 30 days)` for EVENT, `created_at + 30 days` for EXPENSE. When the sender edits the time, `starts_at` and the `expires_at` of PENDING deliveries are recalculated in the same transaction.
- **The status shown to the sender** is separate from the internal state: cancelled by the sender → Cancelled; DELIVERED → Delivered (unchanged even if the recipient later deletes it); PENDING → Awaiting delivery; any other termination (expiry · block · switch to "do not receive" · deletion before delivery) → Expired.
- The recipient's deletion record is `recipient_deleted_at`. If it is set, subsequent edits · cancellations are not sent down to that person.

13.4 Shared Item Payload Schema

```
EVENT:   { "title": "만나기", "timeKind": "EXACT" | "FUZZY" | "ALL_DAY",   // "만나기" = "Meet up"
           "startAt": "2026-10-03T10:00:00Z" | null, "fuzzyTime": "저녁쯤" | null,   // "저녁쯤" = "a
round evening"
           "date": "2026-10-03",            // for ALL_DAY · FUZZY
           "endAt": null, "timezone": "Asia/Seoul", "location": "강남역" | null,   // "강남역" = "Gan
gnam Station"
           "reminderOffsets": [0, -30] | [] }       // [] means use the recipient's default reminde
r
EXPENSE: { "amount": 30000, "currencyCode": "KRW", "category": "의료" | null,   // "의료" = "Medica
l"
           "memo": "병원비" | null, "occurredAt": "2026-10-02T03:00:00Z",   // "병원비" = "Hospital b
ill"
           "bundleItems": ["아이스크림","생수","밥"] | null }   // = ["Ice cream","Bottled water","Mea
l"]
```

The server validates length (title 100 chars, memo 500 chars, location 100 chars), amount (integer from KRW 1 to KRW 100,000,000), time format, and allowed offset values. **Design**

13.5 Backup · Public Events · Notification Settings

```
create table backup_records (
  user_id uuid references profiles(id) on delete cascade,
  entity_type text check (entity_type in ('EVENT','TRANSACTION','TASK','REMINDER','SETTING',
                                          'RECEIVED_COPY','INBOX','TOMBSTONE')),
  entity_id text, payload jsonb not null, updated_at timestamptz not null,
  deleted boolean not null default false,
  primary key (user_id, entity_type, entity_id)
);
create table public_events (
  owner_id uuid references profiles(id) on delete cascade, event_id text,
  kind text not null default 'EVENT' check (kind in ('EVENT','TASK')),   -- tasks use the same tabl
e
  title text not null, time_kind text, start_at timestamptz, date date, fuzzy_time text,
  end_at timestamptz, location text,
  due_at timestamptz, done boolean,     -- for TASK
  updated_at timestamptz not null,
  primary key (owner_id, event_id)
);
```

- `backup_records` : readable only by the owner. **Writes are allowed only through the `backup_upsert` function** (no direct write policy), and the function checks the active device · session. This prevents a replaced old device from overwriting the new device's backup. On upsert, an incoming `updated_at` older than the stored value is ignored.
- `public_events` : writable only by the owner. Readable only when the `can_view_events(owner_id, auth.uid())` function (conditions in 7.1) passes.
- Backup payloads use Postgres · disk encryption at rest (service default). End-to-end encryption is not in the first-release scope. **Design**

13.6 Usage · Trial · Subscription

```
create table entitlements (
  user_id uuid primary key references profiles(id) on delete cascade,
  plan text not null default 'FREE' check (plan in ('FREE','BASIC','PLUS')),
  platform text, product_id text, original_transaction_id text unique,
  period_start timestamptz, period_end timestamptz,
  status text check (status in ('ACTIVE','GRACE','EXPIRED','REVOKED')),
  updated_at timestamptz not null default now()
);
create table usage_counters (
  user_id uuid references profiles(id) on delete cascade,
  bucket text,         -- 'month:2026-10-02T..' (billing period start) or 'day:2026-10-02' (KST)
  ai_used int not null default 0, sends_used int not null default 0,
  primary key (user_id, bucket)
);
create table trials (
  user_id uuid primary key references profiles(id) on delete cascade,
  ai_left int not null default 3 check (ai_left between 0 and 3),
  share_left int not null default 1 check (share_left between 0 and 1)
);
create table always_add_counters (sender_id uuid, recipient_id uuid, day date, n int not null defau
lt 0,
  primary key (sender_id, recipient_id, day));
```

- Deductions are done **atomically within a single DB function**. Since the bucket row may not exist, use the form `insert into usage_counters … values (…, 1) on conflict (user_id, bucket) do update set ai_used = usage_counters.ai_used + 1 where usage_counters.ai_used < limit returning ai_used`. No result means the limit is exceeded. The daily bucket · monthly bucket (subscribers) or `trials.ai_left` (trial) are deducted in the same transaction, and if any one fails, everything is rolled back. Refunds (undoing a deduction) are handled by the same family of functions and can happen only once (keyed by requestId).
- AI deduction rules (4.7): the gateway "reserves" before the call, commits on success (PARSED / including clarifying-question responses), and refunds the user's count on server error · timeout · UNSUPPORTED (it stays counted against the cost budget counter).

13.7 Server Functions (Edge Functions / DB Functions)

Function	Input → Output · Rules
register_device	device_id, platform, push_token → invalidates the previous device. Response: active session id
check_nickname / set_nickname	nickname → availability · reason / 13.1 rules
update_profile_settings	Visibility default, audience_mode, searchable, notifications (all) · lock screen
avatar_upload_url / avatar_commit	Issue a signed upload URL → after upload, commit the version and delete the previous file
create_invite	→ { url, expiresAt(+5 min) }. Previously created unused links are invalidated. Limited to 5 per minute
redeem_invite	token → { result: CONNECTED | EXPIRED | USED | ALREADY_FRIENDS | SELF | BLOCKED | LIMIT, friend }
search_nickname	nickname → 0 or 1 user (nickname · initials only)

send_friend_request / respond_friend_request	On accept, create friend_links
update_friend_settings	alias, 2 receive modes, 2 notification toggles, hidden_all, in_audience, include_expense_in_total
disconnect_friend / block_user / unblock_user / report_user	On block, set links in both directions to DISCONNECTED + cancel pending deliveries
send_item	{ clientRequestId, kind, payload, recipientIds[] } → per recipient { deliveryId | error: NOT_AVAILABLE (not a friend · blocked · not receiving), PAST_EVENT, DAILY_LIMIT, NOT_ENTITLED, TRIAL_USED }. Check order: auth · active device → subscription or trial (trial requires kind=EVENT, 1 recipient) → reject past events → per-recipient relationship · block · receive permission → atomically deduct from the 50-per-day limit **by the number of valid recipients** (if remaining count is insufficient, reject everything; response includes remaining count) → determine routed_mode per recipient (atomically increment the always-add counter) → create rows → push. All in one transaction. Retrying with the same clientRequestId returns the same result
update_item / cancel_item	Sender only. Allowed even if the subscription has expired. version+1; on time change, recalculate starts_at · expires_at. Applied · pushed only to recipients with no deletion record, no block relationship, and a delivery that has not ended. Applied to disconnected recipients (3.7: disconnecting keeps items)
sync	cursor → { deliveries: [new · edited · cancelled received items], friends: [changes], sentStatus: [status changes of sent items], account: {deviceReplaced?, plan}, nextCursor }
ack_deliveries	[{ deliveryId, version }] → record DELIVERED
delete_received / delete_all_from_sender	Record recipient_deleted_at (if delivery is pending, also stop the delivery)
backup_upsert / backup_list / backup_delete_all	Batches of up to 500 / paginated listing / reset (c)
public_events_upsert / delete · friend_events	7.3 · fetch a friend's public events (range up to 3 months)
get_entitlement	→ plan, limits, usage, remaining trial uses, period end date
iap_verify	platform, receipt → store verification → update entitlements
iap_webhook_apple / iap_webhook_google	After signature verification, apply renewal · cancellation · refund · grace period (idempotent: store notification id)
reset_shared (b) / reset_friend (a)	As per the 3.7 table
delete_account	3.7 account deletion: create `change_log` FRIEND_ACCOUNT_DELETED for friends → cancel pending deliveries → delete photo → hold nickname → delete auth user (cascading delete). The response notes that the store subscription is not cancelled

Sync cursor Design

```
create table change_log (
  seq bigserial primary key,
  user_id uuid not null,           -- no FK: so account-deletion events are not cascade-deleted
  kind text not null,              -- DELIVERY_NEW, DELIVERY_UPDATED, DELIVERY_CANCELLED, FRIEND_CH
ANGED,
                                   -- FRIEND_ACCOUNT_DELETED, SENT_STATUS, ACCOUNT_DEVICE_REPLACED,
PLAN_CHANGED
  ref_id text, created_at timestamptz not null default now()
);
```

Every change writes one row per affected user to `change_log` in the same transaction. The `sync` cursor is `seq` (time-based cursors are not used because they can skip concurrent commits). Because commit order may differ from seq order, the response returns only up to "seq values lower than any transaction currently in progress," or re-includes the last few seconds in the next call (duplicates are ignored via version comparison). Rows older than 90 days are purged.

Scheduled jobs Design

- Every minute: **server fallback reminders** — targets are deliveries with `routed_mode = ALWAYS_ADD`, `state = PENDING`, and time kind EXACT. The reminder offsets are the item's `reminderOffsets`, or, if empty, the recipient's default reminder (`profiles.default_reminder_offset`). Select those whose reminder time has passed, within the last 10 minutes, with no send record yet; check the recipient's notification settings (all · per-friend event time · lock screen) and send a remote push. Duplicates are prevented by the send-record table `fallback_pushes(delivery_id, offset, sent_at, primary key(delivery_id, offset))`. When the app saves a received item, it does not schedule reminders that have already passed (prevents double alerts).
- Every 5 minutes: expiry processing (PENDING past `expires_at` → EXPIRED), cleanup of expired invite links.
- Daily: release nickname holds, clean up request fingerprints · logs older than 7 days, clean up finished friend_requests.

13.8 RLS Principles

- All writes that require an active-device check (backup, public events, sending, friend settings) go through functions only. Alternatively, add a policy condition comparing the session id in the JWT with `active_devices.session_id`.
- The default is **deny all**. Direct table writes are allowed only on rows the user owns; writes involving relationships (friend connection, sending, blocking) go only through **security definer functions**.
- Preventing ID-guessing attacks: querying someone else's `shared_items` · `deliveries` by id must return nothing. `shared_items` can be read directly only by the sender and that item's recipients; `deliveries` **only by the recipient**. The sender sees only the display status via the `sent_delivery_status` function.
- `friend_links`: only rows with `owner_id = auth.uid()` are readable. Whether the other person allows receiving from me is exposed only via the `can_send_to(friend_id)` function, which returns just `{ event: true/false, expense: true/false }` (the other person's alias · receive mode · visibility settings are not exposed).
- Never reveal that a block exists: sending to a user who has blocked you always returns the error code `NOT_AVAILABLE` ("보낼 수 없는 상대예요" (You can't send to this person)).

- Public profile fields include the photo version only when the users are friends.
- The service role key is used only inside Edge Functions and is never included in the app.

13.9 Choosing a Payment Verification Approach (Backend Owner's Decision)

Approach	Pros	Cons
Build in-house	No external cost, own the data directly	Must implement Apple · Google verification APIs, server notifications, and grace period · refund handling
RevenueCat, etc.	Many Expo integration examples, simple purchase restore · webhooks	Cost scales with revenue Verify , external dependency

Either way, the `entitlements` table is the source of truth, and the app unlocks features based only on server values.

13.10 Operations · Observability

- Every function logs only the request trace id, result code, and elapsed time. Event titles · memos · amounts · input sentences · tokens are never logged.
- Metrics: per-function request count · error rate · latency, push failure rate, number of PENDING deliveries and average wait time, AI call count · estimated cost.
- Report handling: review `reports` in the Supabase dashboard or a simple admin screen → delete photo · reset nickname · suspend (`profiles.status`). Target handling time 24 hours. Design
- Backups: confirm whether Supabase automatic backups are used and the retention period Verify .

14. AI Gateway and Interpretation Contract

14.1 Current State (verified in the repository's `server/`)

- Node.js 24, server with no external dependencies. Pilot token (`vlm_`) authentication, OpenAI Responses + strict JSON schema, `store: false`.
- Endpoints: `GET /healthz`, `GET /v1/session`, `POST /v1/interpret`.
- Default limits: per user 6 per minute · 30 per day · 1 concurrent; global 500 per day; cost budget of 2,000,000 microdollars per day with 50,000 reserved per attempt → effectively **about 40 calls per day in total**. Daily boundary is UTC.
- Responses support only a **single expense proposal**. A bare "7시" (7 o'clock) is not left to the model; it is handled as a clarifying question.
- Strengths (to keep): atomic reservation before the call, requestId idempotent replay, 7-day fingerprint retention, no request body in logs, validation on both server · client.

14.2 Rationale for Model Choice (gpt-4.1-mini)

This model is the default chosen when the existing gateway (R1) was built, and it is kept for the first release. Reasons:

- **Nature of the work**: the AI in this app performs "extraction" — pulling amounts · dates · titles out of short Korean sentences. No long reasoning is needed; what matters is answering accurately in a fixed JSON format.
- **Strict JSON schema (Structured Outputs) support**: the response format never breaks, keeping server · app validation simple.
- **Not a reasoning model**: responses are fast with consistent latency. This fits the usability requirement that results appear within 1–2 seconds after speaking. Reasoning models are slower and can increase output tokens (cost).
- **Cost**: about KRW 1.3 per call (9.5). Plan pricing assumes KRW 5, leaving a large margin.

Alternative	Pros	Cons · Assessment
gpt-4.1-nano	About 1/4 the unit price (input $0.10, output $0.40 per 1M tokens, verified in official docs)	As a small model, accuracy on colloquial Korean · compound sentences may drop. A cost-saving candidate if it passes the 14.5 acceptance tests
Newer models such as GPT-5 mini	Recommended by OpenAI for complex tasks, may improve accuracy	As reasoning models, may increase latency · cost Verify pricing · latency. Candidate if accuracy is insufficient

The model is a server setting (`OPENAI_MODEL`), so it can be changed without an app update. Before changing it, compare accuracy · latency · cost using the 14.6 acceptance test set and decide. Design

14.3 Changes

Item	Change
Authentication	Pilot token → Supabase access token (JWT) verification. The user id is the JWT's `sub`.
Limits	Based on `entitlements` · `usage_counters` · `trials` (9.1). Daily boundary in KST. Cost budget raised according to subscriber count (operational setting)
Response contract	Multiple actions per 14.4
Time	The model extracts only `hour`, `minute`, `meridiemExplicit`, and relative date. The final time is computed by server code using the 4.4 rules
Deployment location	(A) Keep the current Node server + separate hosting, or (B) port to a Supabase Edge Function. Backend owner decides. (B) reduces components; (A) reuses proven code as-is
Model	Keep `gpt-4.1-mini-2025-04-14`. Switching to another model (e.g., GPT-5 mini) requires passing the 14.6 acceptance tests again

14.4 Request · Response Contract v2 Design

```
POST /v2/interpret
{ "requestId": "uuid", "sourceInput": "sentence (max 1000 chars)", "inputMethod": "TEXT" | "VOICE",
  "now": "2026-10-02T06:10:00.000Z", "timeZone": "Asia/Seoul",
  "followUp": null | { "previousRequestId": "uuid", "actionId": "a2",
                       "question": "몇 시에 만나요?", "answer": "저녁 7시" } }   // "What time are you
meeting?" / "7 in the evening"
// If followUp is present, the server combines the previous request's original text (within server
RAM/fingerprint retention) with the answer and re-interprets only that action.
// If the server does not retain the original text, the app resends the original text in sourceInpu
t (recommended).

200 OK
{ "status": "OK" | "UNSUPPORTED",
  "message": "Korean guidance message when UNSUPPORTED",
  "actions": [
    { "id": "a1", "type": "CREATE_TRANSACTION",
      "state": "READY" | "NEEDS_CLARIFICATION",
      "question": "총 얼마였어요?" | null,   // "How much was it in total?"
      "payload": { "transactionType": "EXPENSE", "amount": 7000, "category": "식비",   // "식비" =
"Food"
                   "memo": "밥", "occurredAt": "…Z", "bundleItems": null } },   // "밥" = "Meal"
    { "id": "a2", "type": "CREATE_EVENT", "state": "READY",
      "payload": { "title": "친구랑 약속", "timeKind": "EXACT", "startAt": "…Z",   // "친구랑 약속" =
"Plans with a friend"
                   "autoMeridiem": false, "location": null, "reminderOffsets": null },
      "dependsOn": [] },
    { "id": "a3", "type": "CREATE_REMINDER", "state": "READY", "dependsOn": ["a2"],
      "payload": { "relativeOffsetMinutes": -60 } },
    { "id": "a4", "type": "SEND_EVENT", "state": "READY",
      "payload": { "recipientsText": ["민수"], "includeInMyCalendar": false,   // "민수" = Minsu (a
friend's name)
                   "event": { …same as CREATE_EVENT payload… } } }
  ],
  "usage": { "aiLeftToday": 19, "aiLeftPeriod": 142, "trialLeft": null } }
```

- Action types: `CREATE_TRANSACTION` (expense · income), `CREATE_EVENT`, `CREATE_TASK`, `CREATE_REMINDER`, `SEND_EVENT`, `SEND_EXPENSE`. Maximum 10 actions per response.

- If `autoMeridiem: true`, the app displays "오후 7시로 저장했어요" (Saved as 7 PM).
- Even after receiving the response, the app must run it through its own validator again before saving. **Send actions are not saved; they open the send confirmation screen.**
- Information sent to the model: the sentence, reference time, time zone, instructions · schema. User id · friend list · records · audio are not sent.

14.5 Rules That Must Be in the Model Instructions

1. Record only expenses actually spent. Do not record negations ("안 썼어" (didn't spend)), plans ("살 거야" (going to buy)), questions, or budget · price inquiries.
2. Bundled expense rules (4.5). If a total is given, record 1 entry; split only when every item has an amount and no total is given. Sum mismatch · missing amount → ask a question.
3. For time, extract only numbers and cues; do not guess AM · PM.
4. "이걸 OO에게 줘 / 보내줘 / 넣어줘" (give / send / add this to OO) is a send action. "나도" (me too), "내 캘린더에도" (in my calendar too), "같이 넣어줘" (add it for both of us) mean `includeInMyCalendar: true`.
5. Location expressions ("강남역에서" (at Gangnam Station)) are separated into `location` and excluded from the title.
6. Choose categories only from the fixed list (identical to the app's category list).
7. If not confident, do not guess; mark that action `NEEDS_CLARIFICATION`.

14.6 Acceptance Tests (Minimum Set)

Input (reference time KST)	Expected result
밥 7000원 (Meal KRW 7,000) (any time)	Handled by local parser, no AI call
밥 7000원짜리 먹었고 오늘 친구랑 오후7시 약속있다 (Had a KRW 7,000 meal and have plans with a friend today at 7 PM) (15:00)	Expense 7,000 + event today 19:00, autoMeridiem false
친구랑 7시 약속 (Plans with a friend at 7) (06:30)	Today 07:00, autoMeridiem true
친구랑 7시 약속 (Plans with a friend at 7) (07:10)	Today 19:00, autoMeridiem true
친구랑 7시 약속 (Plans with a friend at 7) (20:00)	Event NEEDS_CLARIFICATION "내일로 할까요?" (Make it tomorrow?)
내일 3시 치과 (Dentist tomorrow at 3) (any time)	Tomorrow 15:00, autoMeridiem true
내일 9시 회의 (Meeting tomorrow at 9)	NEEDS_CLARIFICATION AM/PM
아이스크림 먹었고 생수 샀고 밥 먹었고 총 12000원 (Had ice cream, bought bottled water, had a meal, KRW 12,000 total)	1 expense of 12,000, 3 bundleItems
아이스크림 2000원 생수 1000원 밥 9000원 (Ice cream KRW 2,000, bottled water KRW 1,000, meal KRW 9,000)	3 expenses

아이스크림 2천원 생수 1천원 밥 9천원 총 15000원 (Ice cream KRW 2,000, bottled water KRW 1,000, meal KRW 9,000, KRW 15,000 total)	NEEDS_CLARIFICATION
내일 오후 7시에 강남역에서 만나기 이걸 민수에게 줘 (Meet at Gangnam Station tomorrow at 7 PM — give this to Minsu)	SEND_EVENT, title "만나기" (Meet up), location "강남역" (Gangnam Station), recipientsText ["민수"], includeInMyCalendar false
내일 저녁 7시 민수랑 저녁, 나도 넣고 민수한테도 줘 (Dinner with Minsu tomorrow at 7 in the evening — add it for me and give it to Minsu too)	SEND_EVENT, includeInMyCalendar true
커피 안 샀어 5000원 (Didn't buy coffee, KRW 5,000)	UNSUPPORTED (not recorded)
점심 8천원 썼고 내일 민수 만나. 한 시간 전에 알려줘 (Spent KRW 8,000 on lunch, and meeting Minsu tomorrow. Remind me an hour before)	Expense READY, event NEEDS_CLARIFICATION (time), reminder dependsOn the event

15. Exception · Error Handling Table

Situation	Handling
Sentence requiring AI while offline	Keep as a draft, "연결되면 [다시 처리]를 눌러 주세요" (Tap [Process again] once you're connected). Not run automatically
Sending while offline	Put in the outbox and show "연결되면 보낼게요" (Will send once connected). clientRequestId is fixed when the item enters the outbox. Sent automatically after reconnecting. Until sent, the sent item shows "보내는 중" (Sending). If it later fails due to limits · relationship · time having passed, the sent item shows "보내지 못했어요(사유)" (Couldn't send (reason)) and one notification is shown
Cancel · disconnect · block · reset · account deletion while offline	Button disabled + "인터넷 연결이 필요해요" (Internet connection required). Exception: deleting a received item on the phone happens immediately, and only sending the deletion record is queued
AI server error · timeout	No deduction. "잠시 후 다시 시도해 주세요" (Please try again shortly) + [Try again] + [Enter manually]
AI response fails app validation (passed server validation)	Not saved. Deduction is kept (4.7). Log an error trace id to improve server validation rules
Speech recognition: no speech · cancelled · permission denied · unsupported	Respectively: "다시 말해 주세요" (Please say that again) / close quietly / permission guidance + open Settings / guide to text input
Notification permission denied	Reminder PERMISSION_BLOCKED, card shows "알림 꺼짐" (Notifications off), Settings guidance
Local notification scheduling fails	Not shown as success; the event is saved · the reminder shows "예약 실패, 다시 시도" (Scheduling failed, try again)
Same expense entered twice in quick succession	Same amount · memo within a short time → "같은 기록이 방금 있어요. 또 저장할까요?" (An identical record was just added. Save again?) (never auto-discard)
Some recipients fail during sending	Show results per recipient (success · reason). [Send again] only for the failed ones
Recipient blocks after sending	Cancel pending delivery; the sender sees the same display as "Expired" (block is hidden)
Trying to send a past event	Sending refused: "이미 지난 일정은 보낼 수 없어요" (You can't send an event that has already passed)
Received item has already passed when saved (expiry boundary)	Add to calendar but with no reminder, status "지남" (Past)
Edit to a received item arrives after deletion	Ignore (deletion record)
Older version of the same item arrives	Ignore (version comparison)
Login on another device detected	"다른 기기에서 로그인했어요" (You logged in on another device) → delete local data → first screen

Pending backups exist at logout	Attempt upload → if it fails, "N건이 아직 백업되지 않았어요. 그래도 로그아웃할까요?" (N items haven't been backed up yet. Log out anyway?)
App closed during restore	Resume restore on next launch (page cursor saved)
Purchase succeeded but server verification failed	Show "구매 확인 중이에요" (Confirming your purchase), retry automatically, guide to [Restore purchases]. Also applied via store webhook
Refund · cancellation notification	Update entitlements, reflected in the app on the next sync
Nickname duplicate · banned word · 30-day limit	Show the reason below the input field, show the next date a change is allowed
Link used simultaneously (two people)	Only one succeeds (DB lock); the other sees "이미 사용된 링크예요" (This link has already been used)
30-friend limit reached	Add button disabled + guidance
50 daily sends reached	"오늘은 더 보낼 수 없어요(자정에 초기화)" (You can't send any more today (resets at midnight))
Push token expired · failed	Server marks token invalid; re-registered on next app launch. Items are received via sync
Excel generation fails (size · permission)	Show an error, suggest retrying with a shorter period
Profile photo upload fails	Keep the previous photo, [Try again]

16. Security · Privacy

16.1 Data Collected · Retained (to be listed in the Privacy Policy)

Item	Purpose	Retention
Social login identifier	Account	Deleted immediately on account deletion
Nickname, profile photo	Friend identification	Deleted on account deletion (only the nickname string is held for 30 days)
Event · expense · task backups	Restore	Deleted on account deletion · reset (c)
Copies of public events	Showing to friends	Deleted when made private · deleted · account deleted
Sent items · delivery records	Sharing	Deleted on account deletion · reset (b). Copies on recipients' phones belong to the recipients
Push token · device info	Notifications · single device	Deleted on logout · account deletion
Subscription transaction info	Payment · refund handling	Verify Legally required retention period under e-commerce law
AI request fingerprints · status (no content)	Duplicate prevention · limits	7 days
Report contents	Operations	A set period after handling (defined by operations policy)

- During AI interpretation, sentences are sent to OpenAI (USA) → **disclosure of processing outsourcing · cross-border transfer** is required Verify . Before the first AI use, the app obtains consent: "입력 문장이 AI 해석을 위해 OpenAI로 전송돼요" (Your input sentences are sent to OpenAI for AI interpretation). Design
- Choose the Supabase data storage region (Seoul region if possible) Verify .
- Raw audio is never stored anywhere.

16.2 Security Checklist

- Only the public key (anon key) in the app. OpenAI key · service role key · store keys only in the server secret store.
- RLS on all tables, relationship writes via security definer functions, block reads · writes using someone else's id (IDOR).
- Re-validate input length · format on the server (do not trust app validation). Do not use client-supplied values such as `sender_id`; use the user id from the JWT.
- Store invite tokens · AI tokens as hashes. Links are single-use · 5 minutes.
- Rate limits: nickname search, link creation, sending, AI, reports.
- No personal content in logs (13.10).
- Prevent Excel formula injection (10.3), strip EXIF from profile photos (5.4).
- Webhooks verify signatures (Apple JWS, Google Pub/Sub authentication).

17. Store Release Requirements

Verify The following are general requirements as of when this document was written; recheck them against the latest Apple App Store Review Guidelines and Google Play policies before submission.

Requirement	Store	This app's response
In-app account deletion	Apple · Google	Settings > Account > Delete account (3.7). Google also requires a web deletion-request URL outside the app → prepare a simple web form
User-generated content: reporting · blocking · handling inappropriate content	Apple · Google	5.2 block · report, 13.10 operational handling, nickname banned words
Offer Sign in with Apple	Apple	Included (3.1)
Digital goods via in-app purchase	Apple · Google	9.4
Auto-renewal disclosure · terms · privacy links · restore purchases	Apple · Google	9.2 paywall
Privacy label / Data safety form	Apple · Google	Filled out based on the 16.1 table (collected: identifiers, user content, purchase history / no tracking)
Permission purpose strings	Apple · Google	Write microphone · speech recognition · photos · camera · notification strings in Korean
Android notification permission · exact alarms	Google	8.2. Document justification for using the exact alarm permission as a calendar · reminder app
Review account · notes	Apple · Google	Two test accounts in a mutual-friend state for testing login-required features (friends · sending), plus guidance for test subscriptions (sandbox)
Age rating	Apple · Google	Complete the questionnaire. Sign-up for ages 14 and over
Closed testing requirement for new personal developer accounts	Google	A closed test with a set number of testers · duration may be required → start early
Universal Links / App Links	Apple · Google	Host association files on the invite link domain (5.1)

18. Development Order · Roles · Completion Criteria

Phase	Scope	Owner	Completion criteria
M0 Preparation	Revise `AGENTS.md` scope rules, Supabase project · region, domain for invite links, apply design mockups per 11.4	App owner · Backend	Revised rules committed, mockups reviewed
M1 Local completion	Events · tasks · reminder executor, manual entry form, AM/PM rule code, local notifications + default reminder, text size, Excel, 12.3 migration	Frontend	`pnpm typecheck` · lint · test pass, notifications confirmed firing on a real device, 200% text screens checked
M2 Accounts · Backup	Apple · Google · Kakao login, nickname, single device, backup · restore · guest data choice (import / overwrite), logout, account deletion, reset (c)	Backend + Frontend	Demo: delete app → reinstall → log in → data · reminders restored; old device cleaned up when logging in on another device
M3 AI expansion	Gateway auth switch, v2 contract, limits · trial · KST, result cards (multiple), bundled expenses	Backend + Frontend	All 14.6 acceptance tests pass, deduction rule tests
M4 Friends · Sharing	Link · QR · nickname, receive settings, send · receive · edit · cancel · deletion records, push, server fallback reminders, public events · viewing friends' events, profile photo, block · report, reset (a)(b), voice sending	Backend + Frontend	Demo all Chapter 6 flows on two real devices (iOS · Android); confirm via API that acceptance status is not exposed to the sender
M5 Subscription	Product registration, purchase · verification · webhooks · restore, paywall, expiry handling	Backend + Frontend	Sandbox purchase · renewal · cancellation · refund · restore scenarios pass
M6 Release preparation	Chapter 17 requirements, privacy policy · terms, store materials, closed testing, performance · accessibility review	Everyone	Submission checklist complete

18.1 Common Completion Criteria

- Type check · lint · tests pass; record changed files · checks run · known limitations.
- Verified on real Android · iOS devices (notifications · push · payments cannot be substituted with simulators).
- No path that displays a failure as a success.
- RLS tests: reading · modifying another user's data with a different user's token always fails.

19. Excluded from First Release · Second-Release Candidates

Item	Status · Reason
In-app chat	Excluded (confirmed). KakaoTalk etc. already exist, and it conflicts with server retention of conversations · review burden · the "acceptance status is private" principle
Recurring events	Excluded from first release, revisit in second Confirmed . Requires per-date deletion records, limits on number of scheduled notifications, and edit-scope rules ("this one only / all future")
Linking my event ↔ sent event	Second-release candidate (first release uses independent copies)
Account linking (Apple ↔ Google ↔ Kakao)	Second-release candidate
Scheduled delivery	Excluded (confirmed) (the event date itself is sufficient)
Local rule parser for events (free voice event entry)	Second-release candidate
Backup merging (guest + server)	Second-release candidate
Annual billing, store free trial period	Second-release candidate
External calendar integration, habits, weekly AI report, advanced search	Out of scope
"카카오톡 등으로 공유" (Share via KakaoTalk, etc.) button	Not included because it was not requested

20. Release Prerequisites · Verification List

20.1 Release Prerequisites (no impact on the design)

Prerequisite	Details
Business registration	Before subscription revenue begins. Via Hometax · tax office. Confirm the type (simplified · general) and whether mail-order business registration is required with the tax office · a tax accountant
Apple Developer Program	$99/year (verify). Individual accounts display the legal name; displaying a business name requires an organization account (D-U-N-S required). Enroll before payment development (M5)
Google Play Console	$25 one-time (verify). Enroll early to prepare for the closed testing requirement
Expo (EAS) account	Build · submission
Kakao Developers app	Kakao login
Domain + simple web pages	Invite links, link association files, privacy policy · terms · web account deletion request page
Contact email	Store listing · report inquiries
App name · icon · screenshots · description	Store listing
Privacy policy · Terms of service	Based on 16.1, legal review recommended

20.2 Verification List (Consolidated)

1. Supabase: scope of Kakao login support, how to force-terminate other sessions, Seoul region, pricing, automatic backup policy, free-plan pausing policy
2. Apple: login policy, account deletion, user-generated content policy, auto-renewal disclosure, Small Business Program commission, 64 local notification limit, developer account cost · type
3. Google: exact alarm permission policy, notification permission, Data safety form, web account deletion requirement, closed testing requirement for new accounts, subscription commission
4. Legal: method for verifying age 14+, cross-border personal data transfer disclosure, transaction record retention period, business registration · mail-order business registration
5. Cost: OpenAI price changes, exchange rate, readjust limits after measuring actual token usage
6. Cost of payment services such as RevenueCat (if chosen)

Appendix A. Full Decision Log

Topic	Decision
AI approach	Use an API (OpenAI gpt-4.1-mini) instead of in-house training; Korean only
Pricing	Basic KRW 2,900 (150/month · 20/day), Plus KRW 7,900 (500/month · 40/day); the word "unlimited" is prohibited
Trial	Logged-in free members: 3 AI uses + 1 event share (1 recipient), lifetime, re-granted on re-signup. None for guests
Payment	Store in-app purchase, server-side receipt verification, restore purchases
Free usage	Simple expense sentences that can be processed locally + manual entry, receiving, viewing friends' events
Login	Apple · Google · Kakao; can start as a guest; nickname set at login
Backup	Server backup for all logged-in users, phone is the source of truth, restored when logging in after deleting the app. Guest data is lost when the app is deleted
Device	One account, one device; phone data deleted on logout (warning for un-backed-up data)
Account deletion	Immediate deletion, two-step confirmation, sent items remain on recipients' phones labeled "탈퇴한 사용자" (Deleted user), ages 14 and over
Reset	(a) one person (b) all shared data (c) phone + all backups
Adding friends	Link (single-use · 5 min · auto-connect), QR (same rules), nickname (exact search · acceptance required). Friend codes dropped. Cap of 30
Display name (alias)	For each friend, freely set a name visible only on my screen (1–20 chars), applied to lists · source labels · notifications · Excel · voice matching; the real nickname is shown alongside in small text
Nickname	2–12 chars of Korean · English letters · digits, unique, banned words, change once per 30 days, released nicknames held for 30 days, search-allowed toggle
Receiving	Per friend · per type: always ask / always add / always view only / do not receive; asked on first connection, "항상 이렇게" (Always do this) checkbox
What the sender sees	Only awaiting delivery / delivered / expired / cancelled. Acceptance status · notification-off status are private
Delivery	Immediate delivery only. Pending items expire at the event time or after 30 days
Edit · cancel	Applied immediately, also overwrites recipient-side edits, "수정됨" (Edited) label, items deleted by the recipient are never revived
Recipient-side deletion	Free to delete; can bulk-delete everything from one person
Expense sharing	Sent by subscribers; received expenses are included in totals by default (toggle)
Abuse prevention	50 sends per day, always-add beyond 20 per day becomes view-only, inbox kept 30 days

Visibility	Only events · tasks can be public (expenses are private), per-item visibility · fully private per friend · selective sharing, most restrictive rule wins, title · time are shared, default chosen at first login
Notifications	Permission requested on first launch (explanation screen first), all · mine · per friend, arrival / event time, default reminder "정시" (at start time), title shown on lock screen (can be hidden), no warning shown to others when notifications are off
Time interpretation	Today: the nearest time after now; if both have passed, ask. Other days: 1–6 o'clock → PM, 7–11 o'clock → ask, 12 → noon. Indicate when auto-decided
Bundled expenses	1 entry if a total is given; split only when every item has an amount and no total is given; mismatch · missing amount → ask; same category → that category, mixed → Other
Compound sentences	Multiple actions per call, saved independently, auto-save + undo
Voice sending	Subscriber feature (free members get a trial: 1 AI use + 1 share), confirmation screen mandatory, recipient matching done on the phone, "내 캘린더에도 넣기" (Also add to my calendar) off by default · can be turned on by voice · changeable in settings, independent copies, same for expenses
Free event entry	Manual entry form (auto-filled from the spoken sentence), no local event parser
AI deduction	Deducted on success; not deducted on error · uninterpretable input
Profile photo	Circular, auto-resized to 512, pinch · drag crop, EXIF stripped, visible to friends only
Text size	70–200%, 14 steps of 10%, slider + [−][+], follow-system switch
Excel	.xlsx with expense · event sheets, period · type selection, free, generated on the phone
Chat · recurring events	No chat, recurring events in second release
Backend	Supabase. This document's design takes precedence over the existing `AGENTS.md` scope rules (to be revised)
Launch	Simultaneous launch on the App Store · Play Store

— End —